

Examensarbete
LITH-ITN-MT-EX--04/065--SE

Modelling Traffic Scenarios for Realistic Air Traffic Control Environment Testing

**Magnus Axholt
Stephen Peterson**

2004-11-19



Linköpings universitet
TEKNISKA HÖGSKOLAN

LITH-ITN-MT-EX--04/065--SE

Modelling Traffic Scenarios for Realistic Air Traffic Control Environment Testing

Examensarbete utfört i medieteknik
vid Linköpings Tekniska Högskola, Campus
Norrköping

Magnus Axholt
Stephen Peterson

Handledare Vu Duong
Examinator Matt Cooper

Norrköping 2004-11-19

**Avdelning, Institution**

Division, Department

Institutionen för teknik och naturvetenskap

Department of Science and Technology

Datum

Date

2004-11-19**Språk**

Language

☐ Svenska/Swedish☒ Engelska/English☐ _____**Rapporttyp**

Report category

Examensarbete

☐ B-uppsats☐ C-uppsats☒ D-uppsats☐ _____**ISBN****ISRN LITH-ITN-MT-EX--04/065--SE****Serietitel och serienummer****ISSN**

Title of series, numbering

URL för elektronisk version<http://www.ep.liu.se/exjobb/itn/2004/mt/065/>**Titel**

Title

Modelling Traffic Scenarios for Realistic Air Traffic Control Environment Testing

Författare

Author

Magnus Axholt, Stephen Peterson

Sammanfattning

Abstract

As air traffic is forecasted to increase, air traffic control software subsequently needs to be more sophisticated. To efficiently push development forward, testing is important in order to determine usability. The tests need to be adapted to fit a particular purpose and carried out with methods that preserve the validity of the results.

This thesis describes an implementation project carried out at the EUROCONTROL Experimental Centre, Bretigny-sur-Orge, France. The purpose of the project is to create an application that enables a user to create datasets of air traffic to be used for these tests. The application allows for manual work or bulk imports from external data sources. Furthermore it compiles scenarios as output datasets intended for prototype air traffic control software developed at Linköping University.

The application design rationale and development process is described. Some time is spent on demonstrating the flexibility of the application and how its usage fits in a bigger picture.

Nyckelord

Keyword

ATC, ATM, 3D, Visualization, Air Traffic Control, Traffic, Eurocontrol, EEC

Upphovsrätt

Detta dokument hålls tillgängligt på Internet – eller dess framtida ersättare – under en längre tid från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns det lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a considerable time from the date of publication barring exceptional circumstances.

The online availability of the document implies a permanent permission for anyone to read, to download, to print out single copies for your own use and to use it unchanged for any non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional on the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its WWW home page: <http://www.ep.liu.se/>

Modelling Traffic Scenarios for Realistic Air Traffic Control Environment Testing

Magnus Axholt
Stephen Peterson

Norrköping, December 2004



Credits: 20
Level: D
Supervisor: Vu Duong
Eurocontrol Experimental Centre
Bretigny-sur-Orge, France
Examiner: Matt Cooper
Department of Science and Technology
Linköping University
Norrköping, Sweden

Abstract

As air traffic is forecasted to increase, air traffic control software subsequently needs to be more sophisticated. To efficiently push development forward, testing is important in order to determine usability. The tests need to be adapted to fit a particular purpose and carried out with methods that preserve the validity of the results.

This thesis describes an implementation project carried out at the EUROCONTROL Experimental Centre, Bretigny-sur-Orge, France. The purpose of the project is to create an application that enables a user to create datasets of air traffic to be used for these tests. The application allows for manual work or bulk imports from external data sources. Furthermore it compiles scenarios as output datasets intended for prototype air traffic control software developed at Linköping University.

The application design rationale and development process is described. Some time is spent on demonstrating the flexibility of the application and how its usage fits in a bigger picture.

Preface

This report is a part of the final examination of the Master of Science Media Technology Engineering Program given at the Department of Science and Technology, Linköping University. This Master's Thesis was conducted at the European Organization for Safety of Air Navigation, EUROCONTROL, in Brétigny-sur-Orge, Paris, France.

The authors would like to extend their gratitude to Vu Duong and Marc Bourgois for their advice and support, Raymond Dowdall for his expertise, Francois Verge for supplying us with data and all of the PhD students at EUROCONTROL Experimental Centre for just about everything else.

Table of Contents

1	INTRODUCTION	1
1.1	Purpose	1
1.2	Method	1
1.3	Target Audience	2
1.4	Report Structure	2
2	BACKGROUND	3
2.1	EUROCONTROL Experimental Centre	3
2.2	Previous Projects	3
2.2.1	Human Factors, Uppsala University	3
2.2.2	Selection Techniques, University of Paris Sorbonne	4
2.2.3	LSAE, University of Linköping	4
3	PRE-STUDY	5
3.1	Context	5
3.1.1	Data Sources	5
3.1.2	Files for Data Exchange	8
3.1.3	Semi Immersive Workbench	9
3.1.4	The Intended Role of TRAMO	9
3.2	Implementation Goals	10
3.3	Development Environment	11
3.3.1	Platform	11
3.3.2	Programming Language	11
3.3.3	3D Graphics	11
3.3.4	User Interface	12
3.3.5	Data Source Parsing	12
3.3.6	Programming Environment	12
3.4	User Interface Design	12
3.4.1	Layout	12
3.4.2	Requirements	14
3.5	Code Structure	15
3.6	Limitations	15
3.6.1	Geographical Region	15
3.6.2	Projection	15
3.6.3	ATC Concepts	15
3.6.4	Performance Data	15
3.6.5	Modification of LSAE	16
3.6.6	Data Amount	16
4	IMPLEMENTATION	17
4.1	Initialization	17

4.2 New Scenario	19
4.3 Import Scenario	25
4.4 Finalize and Run Scenario	27
4.4.1 Add flights	27
4.4.2 Edit flights	27
4.4.3 Delete flights.....	29
4.4.4 Animate scenario	29
4.5 Output Scenario	29
5 TESTS	31
5.1 Functionality Tests.....	31
5.2 User Tests	31
6 RESULTS	33
6.1 Implementation Goals	33
6.1.1 Must	33
6.1.2 Should	33
6.1.3 Preferable	34
6.2 User Interface Requirements	34
6.2.1 Must	34
6.2.2 Should	35
6.2.3 Preferable	36
6.3 Performance	36
6.4 Purpose Fulfilment	36
6.5 Applicability	36
7 DISCUSSION.....	37
7.1 Findings	37
7.1.1 Zooming in Scenario.....	37
7.1.2 Sorting Lists.....	37
7.1.3 Spatial and Temporal Axes.....	38
7.2 Future Development	38
7.2.1 Interaction Model.....	38
7.2.2 Platform	39
7.2.3 New Output Format	39
8 GLOSSARY	40
8.1 Abbreviations	40
8.2 Terminology	42
9 BIBLIOGRAPHY	44

List of Figures

Figure 1 – Conceptual diagram of the environment surrounding TRAMO	5
Figure 2 – The initial sketch of the interface.....	13
Figure 3 – The scenario holds a linked list of airports	17
Figure 4 – The framework after initialization, when only airports are visible.....	19
Figure 5 – The scenario property dialog window	20
Figure 6 – The time manager	20
Figure 7 – The scenario holds a linked list of flights as well as airports	20
Figure 8 – The flight list.....	21
Figure 9 – The flight property dialog window	21
Figure 10 – The flights hold trajectory events in linked lists.....	22
Figure 11 – The trajectory event list	22
Figure 12 – The trajectory event property dialog window	23
Figure 13 – The framework with a number of flights added.....	24
Figure 14 – The import scenario dialog window	25
Figure 15 – A simple import example where max samples = 10 and sample step = 2	26
Figure 16 – Example of how an imported data set may look in the perspective viewport	27
Figure 17 – Moving a trajectory event in the orthogonal viewport	28
Figure 18 – The export scenario dialog window	30

List of Tables

Table 1 – Table of implementation goals for TRAMO.....	10
Table 2 – Table of interfaces in TRAMO	13
Table 3 – Table of dialog interfaces in TRAMO	14
Table 4 – Table of requirements for the graphical user interface in TRAMO.....	15

List of Data and Code Samples

Sample 1 – RAMS exported data	6
Sample 2 – TAAM exported data	7
Sample 3 – XML flight data.....	7
Sample 4 – XML airport data.....	7
Sample 5 – Flights in scenario.dat	8
Sample 6 – Waypoints in waypoints1.dat	8
Sample 7 – Trajectories in airplanes.dat	8
Sample 8 – Variables in ATM.config	9
Sample 9 – Execution command for TRAMO.....	17
Sample 10 – Using the file handler to import airport data	17
Sample 11 – Manipulator function in the scenario that adds an airport.....	17
Sample 12 - The code in the manipulator function that adds an airport to the scenario	18
Sample 13 - Code to obtain the selected flight.....	23
Sample 14 - Code to create new trajectory event and set which flight it belongs to	23
Sample 15 - Code to add the newly created trajectory event to the selected flight	23

1 Introduction

1.1 Purpose

ATC (Air Traffic Control) involves the process of organizing flights so that they can transport people and merchandise safely and optimally through the airspace. Each country regulates its own airspace. In Europe most states are members of EUROCONTROL and use this body to organize international and domestic flights.

Air transports are steadily increasing and the airspace is subsequently becoming harder to regulate. This calls for more sophisticated systems. The research is multi-disciplinary, most of which tends to incorporate computer science. Each new concept contributing to this research should be tested as to assert its importance and future potential.

A Prototype ATM (Air Traffic Management) system is currently being developed by Linköping University and EUROCONTROL. It utilizes computer graphics to convey important information and facilitate the work regarding ATC.

The purpose of this project is to create an application that allows the user to model an air traffic scenario and export it the prototype ATM system. Realistic and flexible air traffic scenarios will make it possible to conduct tests with objective results that will be used to accept or reject suggested future paths of development of this ATM system.

The ATM system will henceforth be referred to as the LSAE (Linköping Semi-Immersive ATC Environment). The LSAE project and EUROCONTROL will be described further in chapter 2.2.3 and 2.1 respectively.

1.2 Method

As in most software development projects a big challenge is to get a firm grip of what is being modelled. The ATM domain is a complex area and to be able to model it and provide a useful tool, the developer must do research on the particular field. In this project, staff at EUROCONTROL provided expertise, in particular Senior Expert Raymond Dowdall.

The research was followed by a phase where the project purpose was broken down into articulated implementation goals. At this point it was possible to make a rough Gantt time schedule. To facilitate communication on report meetings, the project and its application was given a name, TRAMO (TRAffic MOdeller).

Based on the requirements set forth in each goal, means to reach them were decided during the technical pre-study. The pre-study yielded a system design document that described interfaces and class structures.

The development phase was divided into two parts with time for check-up and re-focus in between. Should any part need more work, this was the time to make adjustments to the time schedule or prioritize.

The end of the development phase served as test period as the application gradually became more and more complete. Some time at the end was set aside for validation before eventually performing the final test together with the intended user.

1.3 Target Audience

This report is intended for an audience with a technical background and basic knowledge of software engineering. The target audience is not expected to be familiar with the ATM domain or 3D computer graphics.

1.4 Report Structure

The Introduction chapter explains the purpose of the project. The Background chapter gives examples on who could use the application. The Pre-Study chapter sets the application in a technical context. The Implementation chapter describes how the user is intended to work the application and explains some of the details in the code. The Test chapter describes both tests for software debugging as well as usability tests. The Results chapter summarizes the purpose and the implementation goals. The Discussion chapter explains why some of the goals were not met, how future improvements can be made and how the application is intended to contribute to future research.

All abbreviations in the report are listed in chapter 8.1. Terminology inherit to the ATM and computer science domain is explained in chapter 8.2. Code examples are presented within frames and are intended for an audience with more than basic C++ programming knowledge.

2 Background

2.1 EUROCONTROL Experimental Centre

“EUROCONTROL is the European Organization for the safety of air navigation. This civil and military organization which currently numbers 34 member states has as its primary objective the development of a seamless, pan-European ATM system. The achievement of this objective is a key element to the present and future challenges facing the aviation community, which are to cope with the forecast growth in air traffic, while maintaining a high level of safety, reducing costs, and respecting the environment.” [6]

The EUROCONTROL Experimental Centre [7] is situated in Brétigny-sur-Orge outside Paris in France. Its mission is to carry out research and development in order to improve ATM in Europe. The EEC has several so called research areas, one of which is INO. The responsibilities placed upon the INO [8] department are to coordinate studies on new topics suggested by the ACARE [9] Strategic Research Agenda; to establish an innovative research management framework and to develop creative thinking and skills among the EEC staff.

The EUROCONTROL Experimental Centre outsources the main part of basic fundamental research. The practice is to find use for existing concepts although transferred into and adapted to the ATM domain. This is one of the differences between an experimental center and a research center.

2.2 Previous Projects

EUROCONTROL Experimental Centre’s INO department started its investigation of 3D graphics within ATM four years ago. Almost three years ago the concepts of 3D graphics were developed further and pushed into the domain of VR (Virtual Reality). Experiments have been divided into clearly defined smaller goals in order to establish taxonomy for future work. The first set of experiments mentioned in this chapter is conducted in collaboration with Uppsala University and University of Paris Sorbonne.

There is also an ongoing collaboration with University of Linköping, although this development has a different approach in that it implements several factors at the same time and is guided by developers’ intuition rather than objective experiment data.

2.2.1 Human Factors, Uppsala University

Monica Tavanti is performing doctoral studies at the HCI (Human Computer Interaction) division of the Information Science Department at Uppsala University in collaboration with EUROCONTROL Experimental Centre. The underlying work that bases her thesis “On the Relative Utility of 3D interfaces” consists in part of “Visualization and Interaction On Flight Trajectory in A 3D Stereoscopic Environment” [10], “Empirical Analysis of the Applicability of 3D Stereoscopic in Air Traffic Control” [11] and “Three-dimensional Stereoscopic visualization for Air Traffic Control Interfaces: a preliminary study” [12].

Her studies involved customized applications in the semi-immersive stereoscopic 3D environment. The experiments expanded into the LSAE, see chapter 2.2.3.

2.2.2 Selection Techniques, University of Paris Sorbonne

Nguyen Thong Dang performs doctoral studies at the EPHE (Ecole Pratique des Hautes Etudes) department, University of Paris Sorbonne in collaboration with EUROCONTROL Experimental Centre. In his thesis “A Stereoscopic Visualization Environment for Air Traffic Control – An Analysis of Interaction and A Proposal of New Interaction Metaphors” he investigates new methods for interaction in 3D ATC (Air Traffic Control). In particular he is studying the use of selection metaphors and selection techniques.

His studies also involved customized applications in the semi-immersive stereoscopic 3D environment.

2.2.3 LSAE, University of Linköping

LSAE [13] is an application for ATM in a 3D visualization and interaction semi-immersive display system developed by Marcus Lange, Jonas Hjalmarsson, Matthew Cooper and Anders Ynnerman at the ITN department, Linköping University. It is continuously being developed and features the basic functions handling flights and trajectories, but also functionality for displaying restricted fly zones and weather phenomena and multimodal interaction. Furthermore it features a client-server solution which has the capability of altering the conditions during the course of a scenario [14].

3 Pre-study

3.1 Context

The following chapter places TRAMO in its context, describing input and output data flows and TRAMO as an entity in a network.

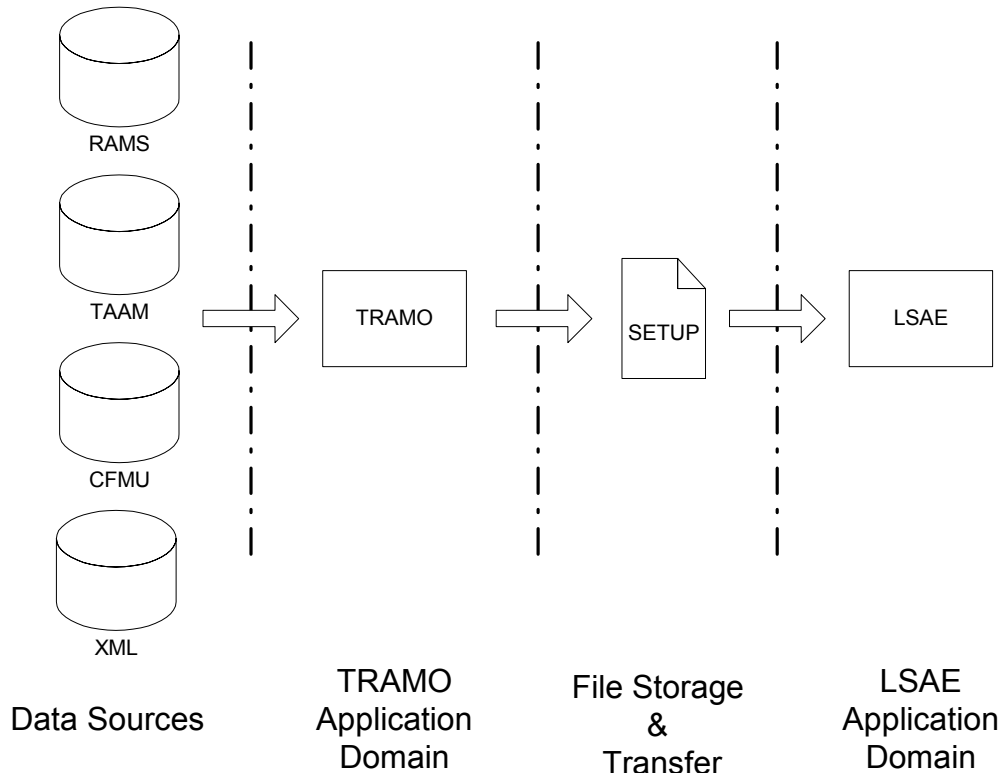


Figure 1 – Conceptual diagram of the environment surrounding TRAMO

3.1.1 Data Sources

The data sources that provide TRAMO with information consist of text files in ASCII 8-UTF format and are either explicitly imported into TRAMO through a menu command or read on application initialization.

3.1.1.1 RAMS Plus

RAMS (Reorganised ATC Mathematical Simulator) Plus [15] is an ATM simulation software developed by ISA Software [16]. ISA Software is a company formed by two former contractors within the EUROCONTROL aircraft performance department. The software performs fast time simulation on aircraft movement from a macroscopic view down to airport level. The software is mainly used for studying the impact of new ATM elements such as introduction of newly restricted airspace zones, runway adjustments and sector management but also for ATCO (Air Traffic Controller) workload analysis.

Typically the RAMS Plus software is fed with information regarding nav aids, traffic schedule, route information, sector coordinates, SIDs (Standard Instrument Departure), STARs (Standard Terminal Arrival Route), holdstacks, runway configuration, taxipaths and

aircraft performance data. This allows the RAMS Plus software to calculate a realistic airspace scenario.

Performance figures can for instance come from the BADA (Base of Aircraft Data) performance database. If RAMS Plus has been provided with proper aircraft performance data this is a guarantee that the specific aircraft type will not execute moves that are non-realistic according to its suggested flight pattern. BADA performance is not the physically possible performance, but rather the recommended flight pattern given economical, technical and various ATM aspects.

The scenario can be exported to a text file format with the file extension *.out. In RAMS terminology this is referred to as RAMS FLIGHTPLAN INITIAL simulation data. The scenario can be set to have a certain discrete sample rate. Selecting a sample rate of six seconds will yield 10 entries per minute for each aircraft present in the scenario. The exported data holds information on the position of each aircraft at each given point of time. Furthermore it holds information on callsign and in what state of the flight the aircraft is in.

```
KEY;SAS6;01:33:30;Cruise;#GCH;FALSE;FALSE;FALSE;430.00;0.00;58.530001;11.70
2225;330.00;330.00;330.00;FlightPhaseEnroute;330.00;60.00;nr;NullSector;Nul
lSector;

KEY;SAS3;01:33:30;Descent;#GCH;FALSE;FALSE;FALSE;300.00;-
2100.00;60.131615;12.260250;140.47;0.00;0.00;FlightPhaseArrival;0.00;50.00;
nr;NullSector;NullSector;

KEY;SAS5;01:33:45;Runway;#GCH;FALSE;FALSE;FALSE;230.00;2600.00;55.852899;12
.578596;97.50;330.00;330.00;FlightPhaseArrival;330.00;70.00;nr;NullSector;N
ullSector;
```

Sample 1 – RAMS exported data

3.1.1.2 TAAM

TAAM (Total Airspace and Airport Modeler) [17] is a software developed by the Boeing company Preston Solutions [18]. It performs fast time simulation on aircraft movement and comprises both macroscopic en-route movements and minute gate events.

In addition to what the previously described RAMS Plus software is capable of, TAAM can also interpolate aircraft movements which render smoother paths and rounded turn radii. Furthermore TAAM allows for very minute adjustments, such as pushback time for a particular aircraft type.

The scenario can be exported to a text file format with the file extension *.gfd. In TAAM terminology this is referred to as TAAM Global Flight Data Recording. The higher level of detail consequently gives more information for export. TAAM exports discreet values with arbitrary interval as sample points describing various functions, one being position over time. The exported data holds information on time, position, altitude, callsign, elapsed time and plenty of additional information. An important fact to realize is that each entry consists of two rows of data.

01,00:09:59	latvlon	MNB452	A30B	1	0	SID	2.64189	49.06262
01,00:09:59	tvalt	AFR321	A343	1	0	ENROUTE	54	26000
01,00:09:59	tvtas	AFR321	A343	1	0	ENROUTE	54	339
01,00:09:59	latvlon	AFR321	A343	1	0	ENROUTE	1.02303	49.81421

01,00:10:05	tvalt	MNB452	A30B	1	0	SID	138	6166
01,00:10:05	tvts	MNB452	A30B	1	0	SID	138	289
01,00:10:05	latvlon	MNB452	A30B	1	0	SID	2.65272	49.06323
01,00:10:05	tvalt	AFR321	A343	1	0	ENROUTE	60	26000
01,00:10:05	tvts	AFR321	A343	1	0	ENROUTE	60	339
01,00:10:05	latvlon	AFR321	A343	1	0	ENROUTE	1.03371	49.80782
01,00:10:11	tvalt	MNB452	A30B	1	0	SID	144	6546
01,00:10:11	tvts	MNB452	A30B	1	0	SID	144	299

Sample 2 – TAAM exported data

3.1.1.3 CFMU

The CFMU (Central Flow Management Unit) [20] provides commercial airlines with flight plans so that the European airspace is utilized as efficiently as possible. To construct the flight plans CFMU calculates a sequence of waypoints from the departure to the destination airport. A waypoint is often marked by a navaid that emits a radio signal used for navigation. Their fixed positions make up the sample points in the planned trajectory.

Filtered CFMU data is available as semi-colon delimited text files.

3.1.1.4 XML

To be able to adjust and test TRAMO, a way to construct arbitrary scenarios would be needed. Therefore an XML (Extensible Markup Language) format was established so that aircraft could be positioned freely over time.

```
<scenario>
  <flight>
    <callsign>SK123</callsign>
    <airplanetype>A320</airplanetype>
    <trajevent>
      <latitude>58.2839</latitude>
      <longitude>18.2903</longitude>
      <altitude>302</altitude>
      <timestamp format="YYYY-MM-DD
hh:mm:ss">2004-08-05 11:42:29</timestamp>
    </trajevent>
  </flight>
</scenario>
```

Sample 3 – XML flight data

This import format is not validated by a performance model, unless it has been exported from simulator software like RAMS or TAAM.

It is also possible to set which airports that should be visible in the scenario.

```
<airports>
  <airport>
    <icao>ESKN</icao>
    <city>STOCKHOLM</city>
    <name>Skavsta</name>
    <longitude>16.917</longitude>
    <latitude>58.783</latitude>
    <altitude unit="ft">140</altitude>
  </airport>
</airports>
```

Sample 4 – XML airport data

3.1.2 Files for Data Exchange

When the LSAE is initiated it reads its scenario information from a set of files. These are:

scenario.dat	Sets departure time for each aircraft
waypoints1.dat	Contains a list of individual waypoints for each aircraft
airplanes.dat	Describes the airplanes in the scenario
ATM.config	Contains variables inherent to design and memory management

It is these files that will serve as an interface for data transfer from TRAMO to the LSAE.

3.1.2.1 scenario.dat

The scenario file states which aircraft that are to be displayed and when they are suppose to take off. The first count is used to allocate the proper amount of memory. Each aircraft is identified by its callsign and its departure time is set in minutes as a delay relatively the scenario start time. SK123 and SK654 take off as soon as the application is started.

```
#This is a comment
FLIGHTS 5
SK123 0
SK456 2
SK789 2
SK321 1
SK654 0
```

Sample 5 – Flights in scenario.dat

3.1.2.2 waypoints1.dat

This file defines the set of waypoints. A waypoint may be redefined but is identified by its ID number. Positions are given longitude, latitude and altitude. Surface coordinates are given in degrees, minutes and seconds. The altitude is set in feet. A set of waypoints define a trajectory. Each entry is one line (excluding comments).

```
#This is a comment
WP 1      lat: 59.00 39.00 50.03  long: 17.00 58.00 44.95  height: 177.00
WP 2      lat: 59.00 40.00 47.60  long: 18.00 6.00 12.30  height: 1420.00
WP 3      lat: 59.00 31.00 54.10  long: 18.00 12.00 12.00  height: 2500.00
```

Sample 6 – Waypoints in waypoints1.dat

3.1.2.3 airplanes.dat

This file describes the behaviour of the aircraft in the scenario. It consists of sections of four rows (excluding comments) like the one displayed below. An aircraft is identified by its callsign. Its movement is determined by either a “arrive” or “departure” command. The type is specified so that LSAE knows which 3D model to use to represent the aircraft. A list of waypoints is matched against a list of time at which the aircraft is expected to arrive at the particular waypoint. The waypoints are described in 3.1.2.2.

```
#This is a comment
airplane SK123 ARRIVE
      TYPE 747
      WAYPOINTS 91 1 2 3 4 5 6 7;
      ETAs      0 0.8 2.8 6.8 9.8 14.8 20.8 23.8;
```

Sample 7 – Trajectories in airplanes.dat

3.1.2.4 ATM.config

This file contains settings that govern appearance, memory allocation and communication. It is differently organized from the other files in the sense that it is not constructed with sequentially ordered records. Instead it can be seen as a set of variables which calls for a search and replace action to be adjusted instead of a simple linear output.

```
#This is a comment
MY_VAR -1

#Number of airplanes to allocate memory for. (1 int)
MAX_AIRPLANES 30

#Number of sectors to allocate memory for. (1 int)
MAX_SECTORS 20

#Number of airplane types to allocate memory for. (1 int)
MAX_AIRPLANE_TYPES 20

#The center position for the camera. (3 floats)
CAMERA_ROTATE_POINT 3578.6 0 1076
```

Sample 8 – Variables in ATM.config

3.1.3 Semi Immersive Workbench

The LSAE is a project that implements a 3D VR system for real time visual representation and manipulation of data in ATC. The LSAE consists of an application with a visual output through a semi immersive workbench displaying a virtual ATC environment.

The screen is a BARCO projection system: BARON 900 with a 136x102 cm monitor which displays a 970x720 px image at 110 Hz. The frame rate depends on the number of objects displayed. The application runs on a Silicon Graphics Onyx2 station with an Irix operating system. The stereoscopic image is created with Stereographics Crystal Eyes glasses with 120 Hz refresh rate equipped with Intersense tracking system IS-900 VWT.

3.1.4 The Intended Role of TRAMO

The intended role for the TRAMO application is primarily to facilitate the generation of traffic in the LSAE. TRAMO will help interpreting and editing the setup files. The secondary role is to give access to external data sources such as TAAM, RAMS and CFMU.

The TRAMO application should import data from one of the data sources or allow a new scenario to be created manually. The scenario will be manipulated and adjusted to fit the particular intended purpose of the flight setup and then be exported to the LSAE. Some of the expected benefits are:

- allow the user to preview the scenario and make adjustments by intuitive “drag-and-drop” technique
- accept data from sources that have the ability to guarantee a realistic flight performance envelope
- accept data from ATC simulation software used by professionals
- enable professionals to export data to the LSAE and thereby increase the validity of the tests performed in the LSAE, since the setup is conceived by subject matter experts and not put through any intermediaries

- eliminate the manual adjustments and specific standards of the various LSAE data files

As mentioned in chapter 2.2.1 Tavanti has conducted tests on human factors. One area within human factors which also has gained a lot of attention within ATM is Situation Awareness. There are a number of tests designed to evaluate Situation Awareness, but many of them have been questioned regarding their validity [21]. One issue affecting validity is the fact that the computerized tests are devised by people who are not subject matter experts of the topic being evaluated. This possible doubt can be removed by letting an ATCO construct a scenario in TRAMO and export it to LSAE for a test on Situation Awareness.

Apart from theoretical academia there is the simple issue of making things as easy as possible. Dang can now position aircraft for selection and interaction for his experiments mentioned in chapter 2.2.2. This will provide a more realistic scenario and also the possibility of evaluating if the previously established research results hold in the LSAE.

The intermediary visualization of TRAMO also provides obvious benefits since the application can function as a dataset viewer. This possibility to import real data and export it into LSAE also provides a range of other possibilities for future tests and possible evolution of the LSAE application itself.

3.2 Implementation Goals

The development of TRAMO was intended to satisfy a set of goals. The importance of each goal is divided into three levels:

- Must – this goal must be met for the application to be usable
- Should – this goal is not entirely necessary, but should be met for the application to be more manageable or useful
- Preferable – this goal is a desired feature, but does not considerably add to the manageability of the application

In the following table, all goals are listed with descending level of importance.

ID	Level	Specification
1	Must	Export data to LSAE from TRAMO
2	Must	Import validated data into TRAMO
3	Must	Trajectory events must have free positioning in 3D and time
4	Must	Create scenario and add flights and trajectory events manually
5	Must	Cover most of the geographical area described in LSAE
6	Must	Modular approach for more manageable code and future development
7	Should	Animate the scenario before export
8	Should	Manageable by an ATCO
9	Should	Display name and location of major airports
10	Should	Display major air routes and their waypoints
11	Preferable	Send scenario data to LSAE from TRAMO via data feed
12	Preferable	Let TRAMO act as an intermediary for data feed from flight simulators

Table 1 – Table of implementation goals for TRAMO

3.3 Development Environment

A number of different decisions were made regarding which tools and APIs (Application Programming Interface) should be used for the development of TRAMO. This chapter briefly describes which important decisions were made, and what factors affected those decisions.

3.3.1 Platform

The platform chosen for development was Linux. The primary reason for this was EUROCONTROL's strive towards wider adoption of open-source software within the organization. This goal is especially prioritized when it comes to high-performance laboratory workstations, the environment where TRAMO is expected to be mostly used.

Secondly, the LSAE runs on IRIX, a UNIX-based operating system. Since Linux is also UNIX-based, it would be possible to make an IRIX version of TRAMO for installation on the same physical machine as LSAE, if the need emerges in the future. However, the software structure has previously been arranged in such a manner that the computer producing the graphics should not be weighed down with peripheral software. The existing client/server setup for controlling the appearance of the LSAE is an example of this approach.

Specifically, all development of TRAMO was made on MandrakeLinux 10. It should however run on any other Linux distribution implementing the 2.4 kernel, as long as the required APIs are installed (see 3.3.3, 3.3.4, 3.3.5)

3.3.2 Programming Language

When choosing language for writing TRAMO, a number of alternatives emerged. Finally C++ was chosen, not because it was necessarily optimal for this specific project, but it met our requirements at the same time as it fulfilled the Implementation Goals (3.2).

C++ is an object-oriented programming language that was developed during the 1980s by Bjarne Stroustrup at AT&T Bell Labs [1], as an extension to the language C. It is one of the most widely used programming languages.

3.3.3 3D Graphics

For the development of TRAMO, there was a need for a graphics library that could render 3D content in real time, and OpenGL was selected for the task.

OpenGL [2] is the industry standard API for displaying 2D and 3D content in computer software. It was developed in 1992 by Silicon Graphics as a platform independent interface, and it is free to use for application developers. However, hardware manufacturers need a license in order to implement OpenGL support in their hardware.

OpenGL exposes a low-level interface for creating 3D content; all models need to be built up through points, lines or by using simple primitives like boxes and spheres. Its main advantage is that the vast majority of graphics hardware has built in OpenGL support. Therefore 3D scenes built up in OpenGL can be rendered very fast, and OpenGL is thus suited for the real time rendering needed in TRAMO.

Additionally, OpenGL has native language bindings for C++, along with C, Java, Ada and Fortran.

3.3.4 User Interface

In order to make development as efficient as possible, a GUI toolkit was needed to rapidly build an intuitive user interface. The basic requirements for this API were the following:

- no license fee
- extensive documentation
- many examples
- support for drawing in 2D
- support for OpenGL integration
- compatibility with C++
- built-in mouse and keyboard event handling

Two alternatives were evaluated: wxWidgets [3] and Qt by Trolltech [4]. Both met the above technical requirements of this project. However, Qt is free only for academic and educational use. If TRAMO will be further developed, and possibly deployed commercially, a commercial license must be purchased. If it stays strictly academic, still an academic license agreement must be established. Therefore wxWidgets was chosen – it is pure freeware.

Moreover, as one of wxWidgets' main advantages is multiple platform support, it is relatively easy to later deploy a version of TRAMO for other platforms such as Microsoft Windows or Mac OS.

3.3.5 Data Source Parsing

The widely spread XML standard makes it easy to find a generic XML parser. XML Wrapp 0.5 [19] is C++ library which is based on libxml2. It organizes the data in a so called DOM (Document Object Model) which is familiar and easily used by most programmers. The XML structure is based on a node tree structure. Such a structure is well suited for translation into an object oriented environment as each node can be interpreted to hold an object with all its attributes.

Hence, the reading of the data can be left for an external generic parser so that only the actual interpretation into object structure needs to be custom made.

3.3.6 Programming Environment

EMACS was chosen as the programming environment. It has many built-in features, as well as helpful code structuring and color-coding functions. It is also pre-installed on most Linux distributions, including the current MandrakeLinux 10.

3.4 User Interface Design

The user interface is a central part of TRAMO, and should be designed to be as intuitive and user-friendly as possible. The interface design process was initiated with the interface layout specification. Later, the detailed requirements for each interface component were specified.

3.4.1 Layout

The layout process started with simple pen-and-paper sketches. These sketches were later refined and compiled into the computer drawn sketch below.

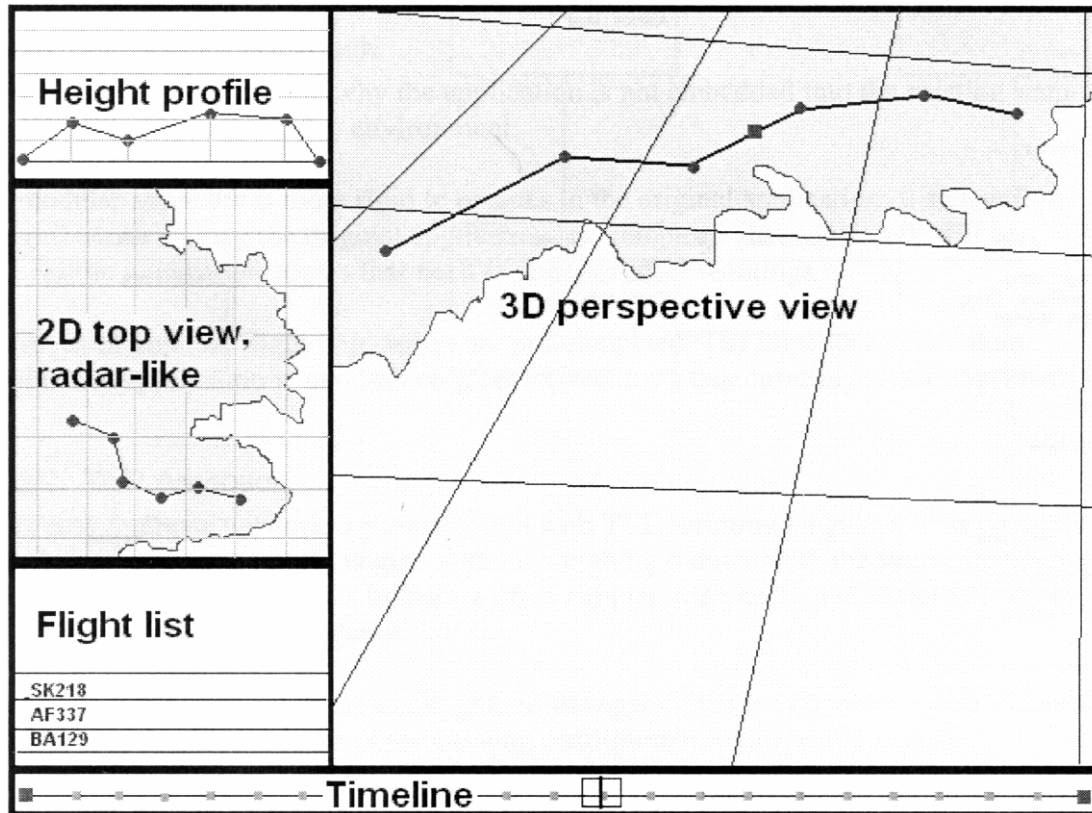


Figure 2 – The initial sketch of the interface

The sketches resulted in the following list of main interface components, which together make up the user interface in TRAMO.

ID	Interface name	Comment
I1	Framework	The main window that holds all other interface components
I2	Time manager	The time slider and animation controls
I3	Flight list	The list of all flights included in the scenario
I4	Trajectory event list	The list of all trajectory events connected to the selected flight
I5	Orthogonal viewport	The 2D window that displays the scenario from an orthogonal top view
I6	Perspective viewport	The 3D window that displays the scenario from a perspective view
I7	Height profile	The 2D window that displays a specific flight's height profile over time from an orthogonal side view

Table 2 – Table of interfaces in TRAMO

In addition to these main interface components, a number of dialog windows were considered necessary. These would allow the user to input numerical and textual data on different entities, and perform other interaction.

ID	Dialog name	Comment
D1	New scenario	Create a new scenario
D2	Open scenario	Open a saved scenario from file

D3	Save scenario	Save the current scenario to file
D4	Import scenario	Import a scenario from an external data format
D5	Export scenario	Export a scenario to an external data format
D6	Scenario properties	Edit the properties of the current scenario
D7	Flight properties	Edit the properties of the current flight
D8	Trajectory event properties	Edit the properties of the current trajectory event

Table 3 – Table of dialog interfaces in TRAMO

3.4.2 Requirements

For the interfaces and dialogs specified in the previous chapter, the requirements were defined and categorized in three requirement levels.

- Must – this requirement is critical and must be met for the application to be usable
- Should – this requirement is not critical, but should be met for the application to be more manageable
- Preferable – this requirement is a desired feature or add-on, but does not considerably add to the manageability of the application

In the following table, all requirements are listed with the requirement ID, interface/dialog ID, requirement level and specification.

ID	Interface	Level	Specification
1	I1	Should	When window size changes, individual interfaces are resized automatically
2	I1	Preferable	Contain debug window for displaying any alerts or error messages
3	I1	Preferable	Resize individual interfaces by sliding splitter controls between them
4	I2	Must	Update aircraft positions in I5, I6, I7 when time slider moves
5	I2	Must	Show actual time of the slider position
6	I2	Should	Contain buttons for starting and stopping animation
7	I3	Must	Contain buttons for adding, editing and deleting flights
8	I3	Must	Toggle trajectory visibility in I5, I6
9	I3	Should	Sort alphabetically
10	I4	Must	Sort according to trajectory event date/time
11	I4	Should	Contain buttons for adding, editing and deleting trajectory events
12	I5	Must	Possibility to scale and translate the scenario
13	I5	Must	Show grid and corresponding latitude and longitude coordinates
14	I5	Should	Contain a function to default the view if user gets lost
15	I5	Should	Possibility to edit trajectory events
16	I5	Preferable	Show waypoints and air routes
17	I5	Preferable	Possibility to add and delete trajectory events
18	I6	Must	5 DOF (no roll) for translation, rotation and scale of the scenario
19	I6	Should	Contain a function to default the view if user gets lost
20	I6	Should	Show grid and corresponding latitude and longitude coordinates
21	I6	Should	Show a rectangle representing the area currently visible in I5
22	I7	Must	Possibility to edit altitude of each trajectory event

23	I7	Should	Ability to scale horizontally and vertically
24	I7	Should	Contain a function to default the view if user gets lost
25	I7	Should	Possibility to change altitude of trajectory event with mouse drag-and-drop
26	I7	Preferable	Possibility to add and delete trajectory events

Table 4 – Table of requirements for the graphical user interface in TRAMO

3.5 Code Structure

The following chapter describes the structure of the code. The classes can generally be divided into three types:

- classes that handle the internal data representation
- classes that present information to the user and handle interaction in the graphical user interface
- classes that perform input/output data operations

A complete class description is found in Appendix 2. The UML (Unified Modeling Language) diagram in Appendix 1 will give an overview of what the classes contain and how they are interconnected. For specifications on wxWidgets' classes (starting with "wx"), please refer to wxWidgets' online specification [5].

3.6 Limitations

3.6.1 Geographical Region

Scenarios are currently limited to a subset of the geographical region one shown in LSAE. The region stretches between 54.8N - 66.5N latitude and 4.5E - 28.0E longitude, which roughly includes southern Scandinavia and Finland.

3.6.2 Projection

The world map in TRAMO is based on the Mercator projection, which is regular around the equator and increasingly rectilinear towards the poles. However, due to the fact that only a smaller region on the earth is displayed, earth's curvature will be discarded and the map texture will be displayed as a uniform rectilinear grid.

In effect this means that the map is displayed on a plane instead of a sphere, with objects drawn according to regular distances not using compensating transformations.

3.6.3 ATC Concepts

Air traffic control sectors will not be implemented in TRAMO at this stage. Sectors are the three-dimensional volumes that divide the airspace into manageable areas of responsibility for different ATCC (Air Traffic Control Center) and individual ATCOs. They are not considered as important for this application, since the focus in LSAE is currently not responsibility, but rather manageability.

3.6.4 Performance Data

No performance data module will be implemented. Performance data gives information about the physical abilities of a specific aircraft type. Thus, there is a possibility that the flights

created manually in TRAMO may not be optimal, or even physically possible, from an aircraft performance perspective.

However, imported flight data is often validated in a performance module in the source software.

3.6.5 Modification of LSAE

This project only involves producing new code, not updating or modifying any existing code. Introducing changes in the LSAE source code will require more time for pre-study as there will be two sets of code standards, two conceptual structures and more possible sources of errors to take into account when debugging.

3.6.6 Data Amount

TRAMO has no relevant limitation on the number of flights or trajectory events it can process. The limit is reached at a point where the data clutters the screen, instead of being interpretable and usable.

TRAMO adjusts the ATM.config configuration file to allocate enough memory in the LSAE application. However, each flight is associated to a number of waypoints and the amount of memory allocated for waypoints cannot be set in a configuration file, but is rather hard coded in the LSAE. Currently, this number is 400. It is impossible to change without recompiling the LSAE source code – which is out of scope of this project.

To summarize, it is possible to have as many flights and as detailed trajectory events as the human perception allows, but the LSAE code structure limits the amount of possible waypoints and effectively also the number of flights.

4 Implementation

4.1 Initialization

The TRAMO application is executed through a Linux terminal window.

```
./Tramo
```

Sample 9 – Execution command for TRAMO

This command will create an empty scenario and start the framework. To simplify code structure all internal data is held and organized in the scenario. The framework is used to display the data in the scenario and therefore manages user interface functions.

While the framework is loading it reads an XML file with data on the airports that will be visible in the scenario. The scenario has a separate list for this and uses some simple functions to manipulate this list. This exemplifies the approach used throughout the code: The scenario holds all data which is broken down in logical groupings, accessible through a set of manipulator functions.

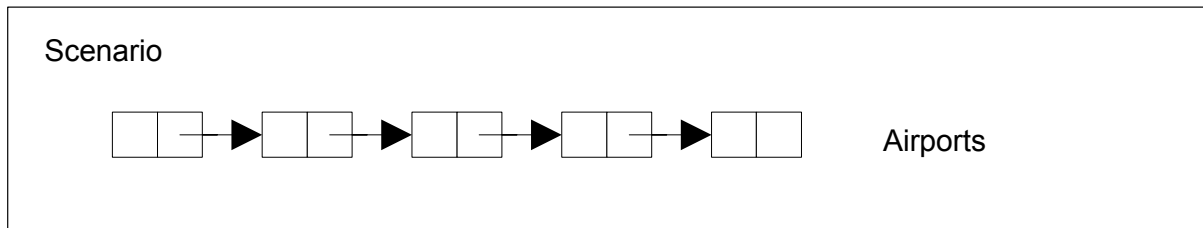


Figure 3 – The scenario holds a linked list of airports

To load the XML file a file handler is needed. It uses a function that accepts the scenario into which the airports should be loaded, the file where to find the airports, how many that should be read and if the function should report its progress to the terminal window.

```
FileHandler *fh = new FileHandler();  
fh->LoadXmlAirportSet(wxGetApp().GetScenario(), "airports.xml", -1, false);  
delete fh;
```

Sample 10 – Using the file handler to import airport data

Inside the file handler each entry in the file is parsed and converted into airport information. The airport is then added to the scenario with a manipulator function.

```
objScenario->AddAirport(objAirport, verbose);
```

Sample 11 – Manipulator function in the scenario that adds an airport

The manipulator function encapsulates the code that manages the linked list and its nodes.

```
bool Scenario::AddAirport(Airport * airportObject, bool verbose){
```

```

bool airportFound = false;
wxString name = airportObject->GetName();

for (AirportList::Node *node = airportList.GetFirst(); node; node =
node->GetNext()) {
    Airport *airportListNode = node->GetData();
    if (name.CompareTo(airportListNode->GetName()) == 0 &&
name.CompareTo("Undef") != 0) {
        airportFound = true;
        break;
    }
}

if (airportFound) {
    if (verbose) {
        std::cout << "MESSAGE: Scenario::AddAirport has not added " <<
airportObject->GetName() << " to scenario. The airport already exists in
list.\n";
    }
    return false;
}
else {
    airportList.Append(airportObject);
    if (verbose) {
        std::cout << "MESSAGE: Scenario::AddAirport has added " <<
airportObject->GetName() << " to scenario.\n";
    }
    return true;
}
}

```

Sample 12 - The code in the manipulator function that adds an airport to the scenario

This may at first seem like a tedious way of organizing the code, and this might very well be the case if the code was to be used only once at a particular place. However, the data structure becomes quite intertwined after a while and this calls for a good conceptual code structure. Manipulator functions control access according to object oriented programming principles of encapsulation. Once the detailed coding is done, the aim is to only expose trivial calls for functions instead of large amount of error prone code. This is what the code above aims to illustrate. The first set of lines exemplifies how simple coding is when one does not have to bother with details.

By the time the airports are read in to the data structure the main interface has loaded and will update the orthogonal and perspective viewports.

In the perspective viewport the world is visualized as a large plane that has been covered with a map texture. All items present in the scenario are drawn from a separate render function which is called after interface initialization and after each user interaction. Initially there are only airports to be drawn. The render function iterates the airport list in the scenario and uses the scenario origin and the airport longitude, latitude and altitude coordinates to construct a transformation matrix which is used to position the airports correctly. In this viewport airports are drawn as red spheres. The altitude is exaggerated somewhat to increase visibility and enhance altitude perception.

In the orthogonal viewport there are only two dimensions and the airports' latitude and longitude coordinates are scaled and used for placing airport icons (red circles) on the map.

It is possible to navigate around in the perspective and orthogonal viewports. In the perspective viewport, the user pans around the scenario by pressing the left and right arrows on the keyboard. To zoom in and out, the up and down arrows are used. In the orthogonal viewport, the user pans in the x and y directions by using the same arrow keys. To zoom, however, the plus and minus keys on the keyboard are used.

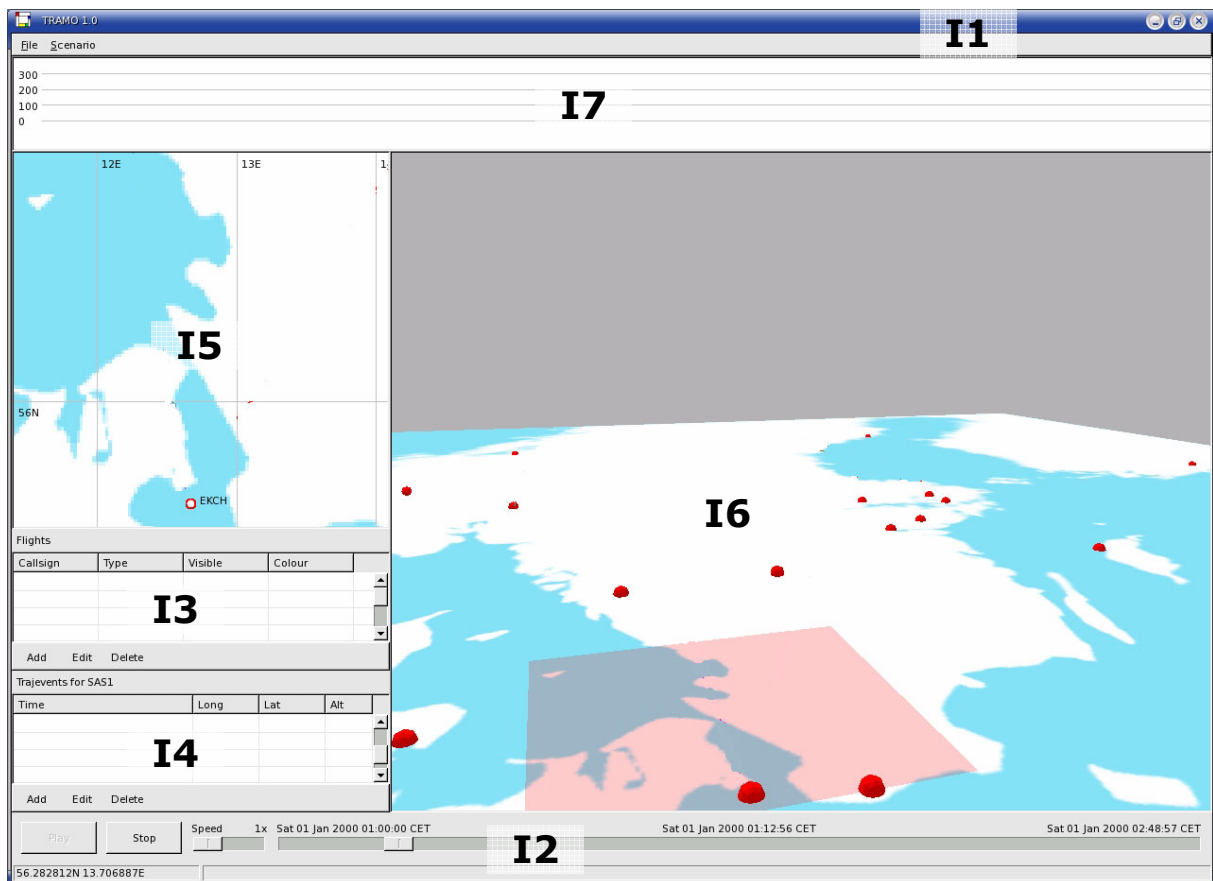


Figure 4 – The framework after initialization, when only airports are visible

When the framework has finished loading, and the airports are drawn in the orthogonal and perspective viewports, it will look like the above image. In the image, each interface is labeled with its ID from the pre-study chapter 3.4. Each individual interface, and how they interact, will be described in more detail in the following chapters.

Now the user has the option of either constructing a new scenario manually or to import a scenario from an external data source.

4.2 New Scenario

Selecting a new scenario from the menu effectively erases any data structure that was linked to a possible previous scenario. The list of airports is however saved to the new scenario since airports are very useful reference points in a scenario.

In the scenario property dialog window that appears (see image below), the user enters the time over which the scenario is supposed to be played out. This can be anything from a couple of minutes up to days.

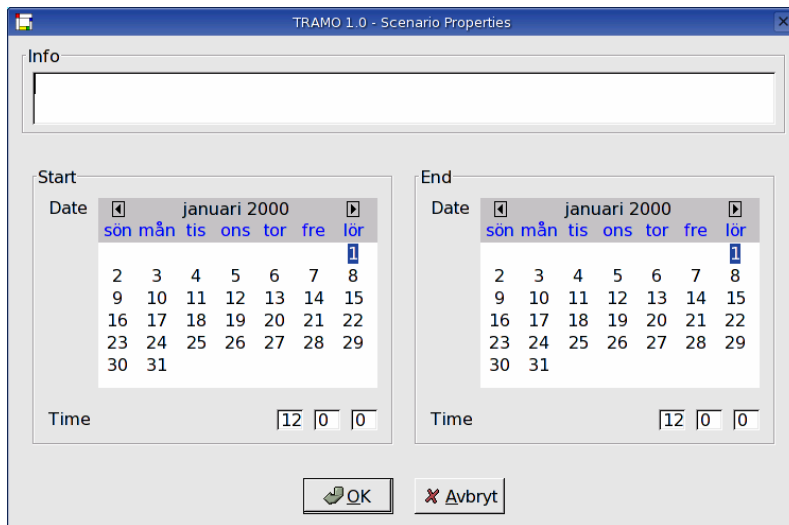


Figure 5 – The scenario property dialog window

As soon as the scenario has a time reference the time manager is loaded with new values. The time manager displays a start and end time. Its slider position reflects the current time. The user will utilize its functions when manipulating the scenario as described in chapter 4.4.



Figure 6 – The time manager

Next step for the user is to create flights and make them fly along trajectories. As previously mentioned all this data is kept within the scenario data structure. The revised conceptual image of the scenario is illustrated below and is represented as a list of flights as well as a list of airports.

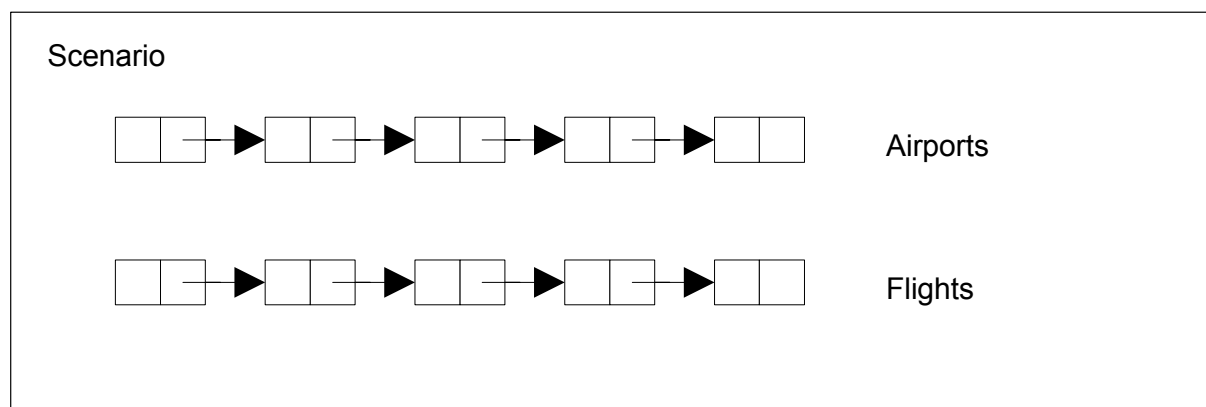


Figure 7 – The scenario holds a linked list of flights as well as airports

In order to create a new flight the user interacts with the flight list, clicking the Add button.

Flights				
Callsign	Type	Visible	Colour	
SAS908A_A	B763	Yes	BLUE	
VKG817_A	DC10	Yes	BLUE	
SAS908B_A	B763	Yes	BLUE	
NWA98_A	DC10	Yes	BLUE	
EUL2212_A	B738	Yes	BLUE	
WIF8331_A	DH8C	Yes	BLUE	
Add Edit Delete				

Figure 8 – The flight list

Clicking the Add button will open the flight properties dialog window. The user can set callsign and aircraft type. The callsign is what uniquely identifies a flight. If there already is a flight in the flight list that has the same callsign, the application will display an error message. There is no need to fill out the departure and destination airports, but if the user chooses not to do so, the flight will be considered to be en-route over the scenario region. The airports are uniquely identified with their ICAO (International Civil Aviation Organization) code and must be in the XML file that was read by the application on start-up. Invalid ICAO codes yield an error message. The visibility settings are meant to help the user if there are many flights present in the scenario. Flight trajectories can be switched on and off and also colored.

Figure 9 – The flight property dialog window

A flight is made up of at least one trajectory event and two airports or at least two trajectory events. The user can increase the level of detail on a flight's trajectory by adding more trajectory events. There need not to be any fixed time space between the trajectory events. In that sense it lacks similarities to a sample point in traditional meaning, but except for this fact it can be considered as a discrete way of representing a continuous movement.

The data regarding a flight's trajectory is saved as a list of trajectory events within the flight stored in the scenario. The user's new mental image of the scenario is now best described as below.

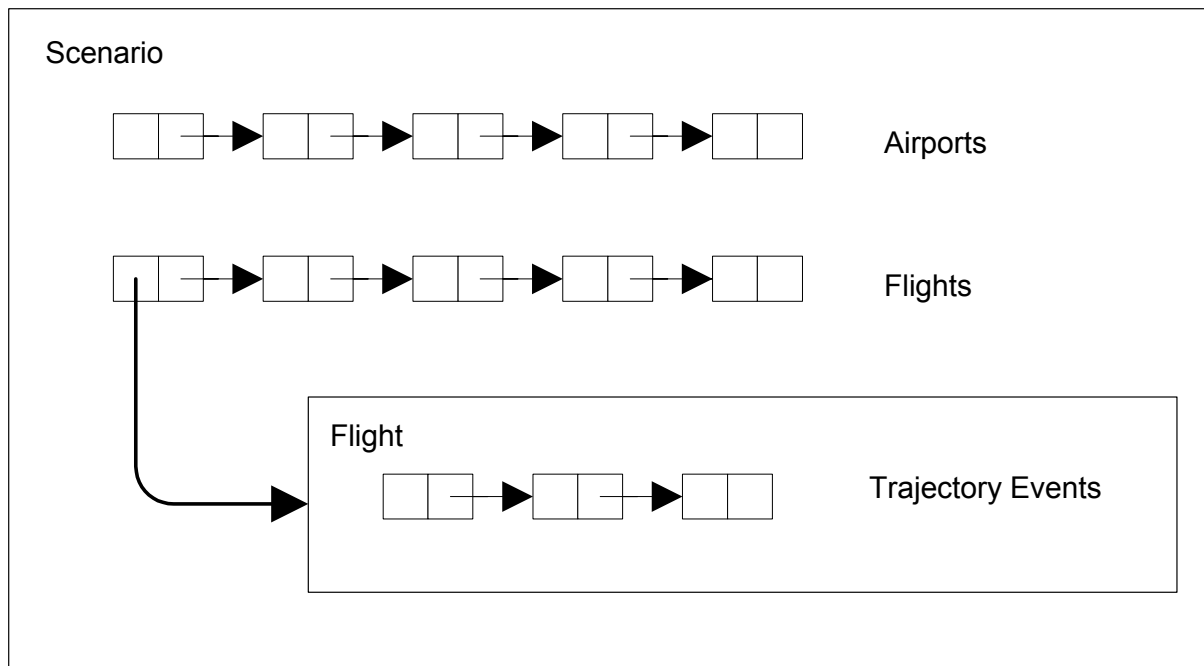


Figure 10 – The flights hold trajectory events in linked lists

The user adds trajectory events to a flight by selecting the flight in the flight list and clicking the Add button on the trajectory list.

Trajevents for FIN651N_A			
Time	Long	Lat	Alt
Sun 02 Jan 2000 01:58:59 CET	11.714024	60.294369	12400
Sun 02 Jan 2000 01:59:59 CET	11.704449	60.390362	10400
Sun 02 Jan 2000 02:00:59 CET	11.647990	60.459656	9200
Sun 02 Jan 2000 02:01:59 CET	11.573743	60.523251	7200
Sun 02 Jan 2000 02:02:59 CET	11.483252	60.575432	5333
Sun 02 Jan 2000 02:03:59 CET	11.358393	60.564312	4222
Add	Edit	Delete	

Figure 11 – The trajectory event list

The Add button displays the trajectory event properties dialog window.

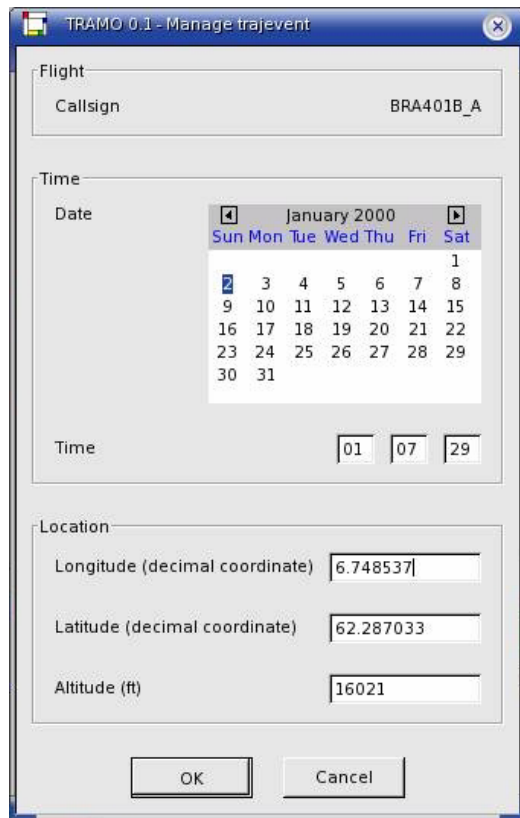


Figure 12 – The trajectory event property dialog window

There can only be one trajectory event at each time instance for a flight. Furthermore, each added trajectory event is immediately sorted into the trajectory event list in chronological order. This way there is no need for a sorting algorithm since the lists are always kept sorted. A sorting algorithm would be rather time consuming and would cause delays in the user interface.

The trajectory event is the smallest entity in the conceptual data structure. This does not pose much of a problem since a solid ground work has produced a stable code structure with nice and simple manipulator functions.

```
Flight *flSelected = framework->GetSelectedFlight();
```

Sample 13 - Code to obtain the selected flight

```
Trajevent *teSelected = new Trajevent();
teSelected->SetFlightCallsign(flSelected->GetCallsign());
```

Sample 14 - Code to create new trajectory event and set which flight it belongs to

```
flSelected->AddTrajeventByTime(teSelected, true);
```

Sample 15 - Code to add the newly created trajectory event to the selected flight

When multiple flights have been added, with a number of trajectory events for each flight, the framework could look something like the image below. In the perspective viewport, the user looks towards the north east, with Copenhagen in the foreground and Umeå in the background. The red square signifies that the orthogonal viewport is focused around

Copenhagen. The flight list is filled with all flights in the scenario. The first flight is selected, so the trajectory event list is filled with its trajectory events, and the height profile of the flight is visible in the height profile window. In the perspective viewport, orthogonal viewport and height profile each visible flight is represented graphically as a number of round icons (the trajectory events) interconnected by a line segments. Each line segment and trajectory event icon has the color previously selected for the flight in the flight property dialog window.

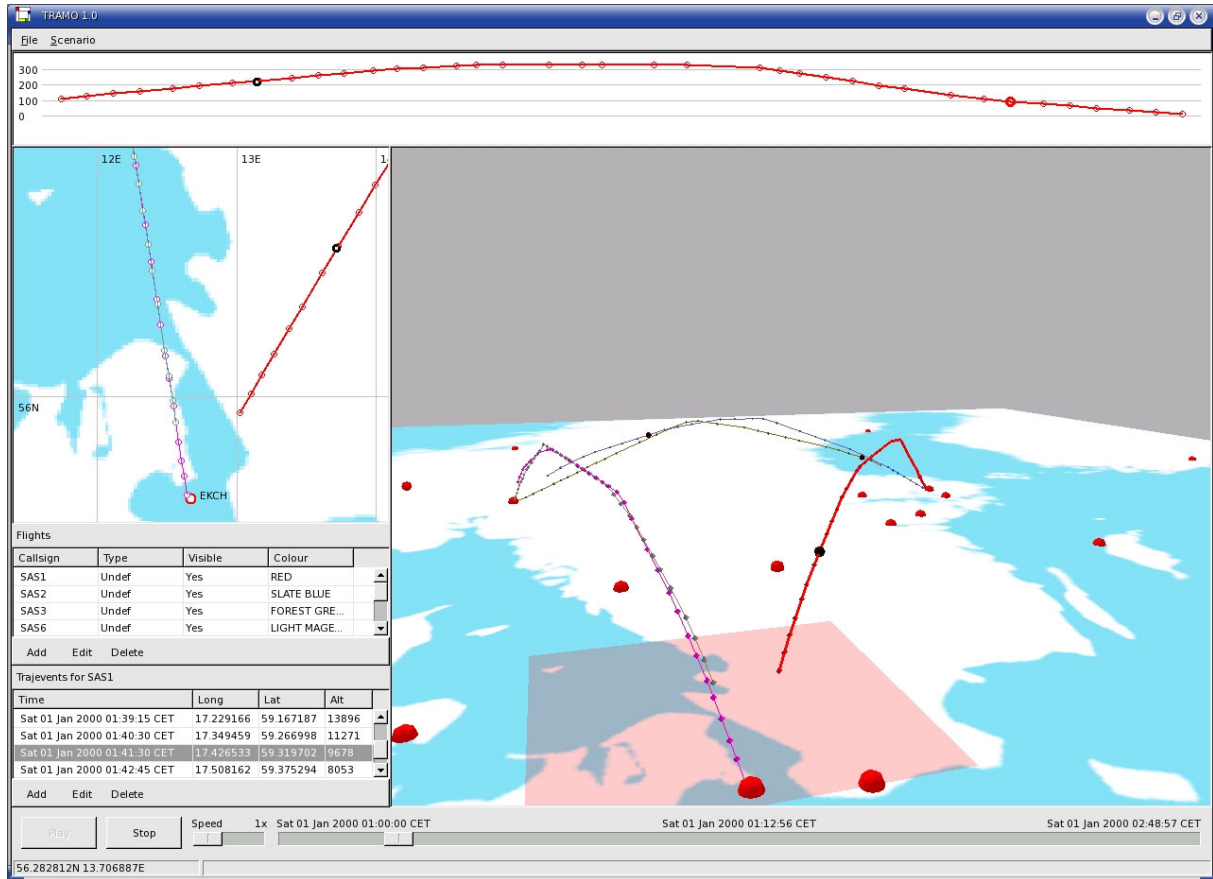


Figure 13 – The framework with a number of flights added

To facilitate the work of the user in constructing a scenario in general, and the flight trajectories in particular, the ambition was to display the existing air route network as an overlay on the ground map texture. This would allow the user to place trajectory event on and in the vicinity of an existing air route and thereby create a somewhat realistic traffic pattern.

The locations of the waypoints that comprise the air route network are listed in each country's AIP (Aeronautical Information Publication). However, a connectivity list describing how the waypoints are interconnected is not (other than graphically on maps). This problem could have been solved by a triangulation algorithm to provide the basic structure as most waypoints are connected to its nearest neighbor. There would however not be a guarantee that this would be the actual air route network, but only something structurally resembling it. Another approach would be tedious manual work to reconstruct the real air route network. Despite the apparent benefit for the user, the time estimated to implement a useful air route guidance system was not justified.

4.3 Import Scenario

Instead of creating a scenario manually by adding flights and trajectory events, there is the possibility of importing a scenario from an external data source. As with creating a new scenario, this action erases any data structure that was linked to the previous scenario. The list of airports is however saved because no data about airports is available in the imported data, and they are very useful reference points in a scenario.

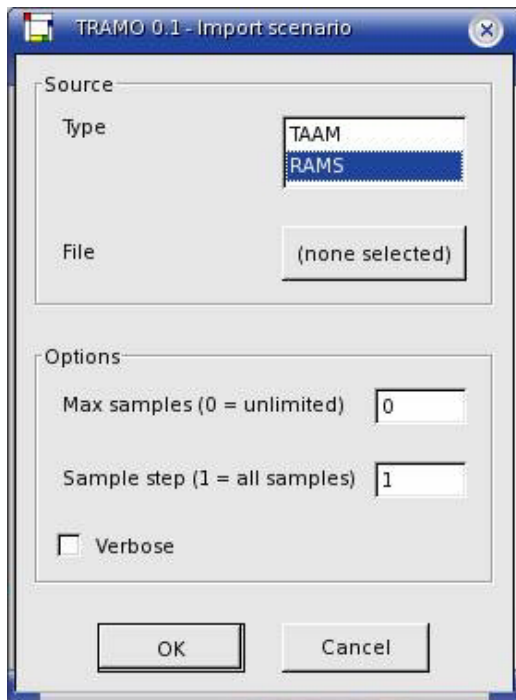


Figure 14 – The import scenario dialog window

In the import scenario dialog window that appears, the user selects which type of data to be imported, together with the specific file that contains the data. Additionally, the user can enter an integer representing the maximum number of entries (i.e. flight trajectory events) that should be read from the file, and another integer representing the sample step within those entries. Finally the user can select if the import should be verbose. If verbose mode is chosen, the import function will print information in the terminal window for each entry imported. This is especially useful for debugging purposes.

Only data from RAMS or TAAM can be imported. XML data is not validated and is only used for constructing sample scenarios for debugging purposes. CFMU data requires too much database processing in order to find the specific position of a flight at a particular time instance and has therefore also been excluded in the import scenario dialog window.

The reason why CFMU data was discarded is because it is based on flight plans that are described as a set of waypoints for a flight to pass. This does not provide enough detail to construct a complex scenario. A waypoint has fixed coordinates, but they are placed far apart and furthermore there is no guarantee that the flight passes directly above the waypoint. Also, the data does not describe a flown trajectory, but a planned route, which in essence means that it has no correlation to aircraft performance.

To deal with these issues CFMU data must be linked to radar data. Radar data is a recording of how the flight actually followed its designated flight plan. This data will, just like the simulator software data, provide exact 3D coordinates on the flight's position at a specific time. However, the radar data is bulky, hard to obtain due to security issues and cumbersome to link to CFMU data.

Below is an illustration of the import flow, where the available data is in the leftmost column, and the actual imported flights are in the rightmost column. The data amount is narrowed in two steps, firstly by selecting the maximum number of entries read, and secondly by selecting the sample step interval. If the time interval between each event in the available data is 6 seconds, and the user selects a sample step of 2, the imported flight data will have a sample time interval of 12 seconds.

When the data has been imported, the start date/time of the scenario is set to the date/time of the first event being imported. The scenario end date/time is set to the date/time of the last imported event. This way, an imported scenario can be animated directly as described in chapter 4.4.4.

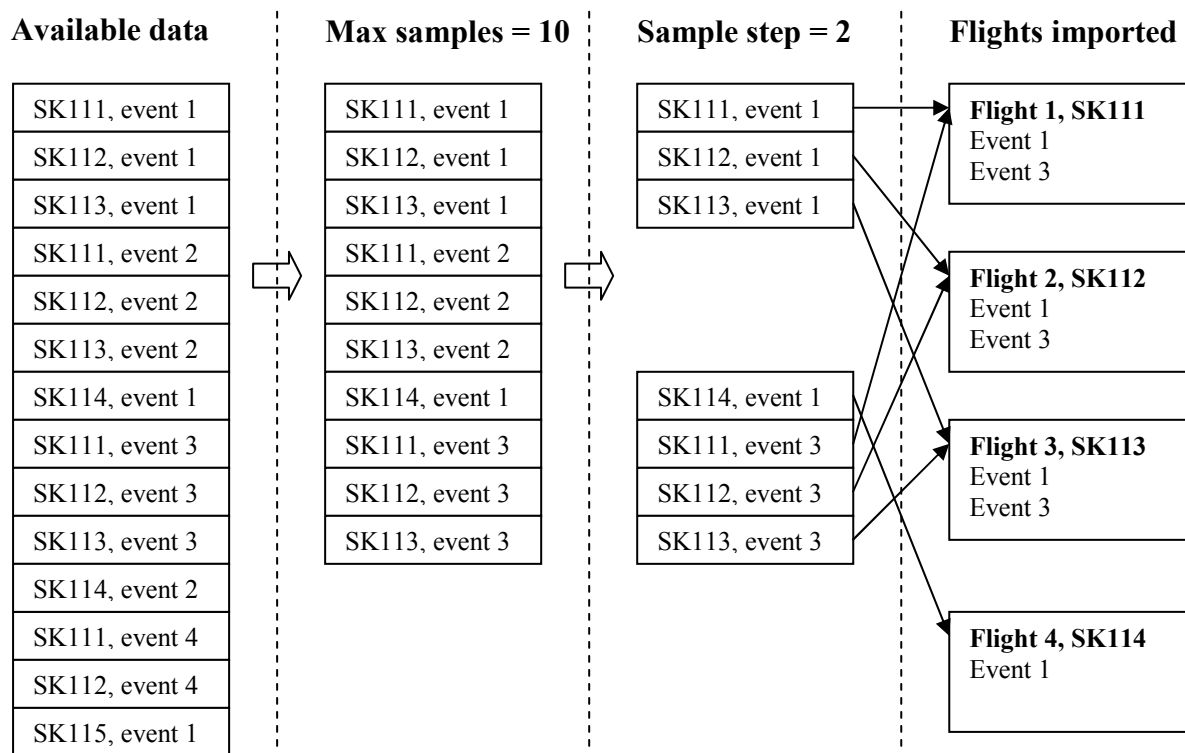


Figure 15 – A simple import example where max samples = 10 and sample step = 2

A large imported data set may look quite complex. Below is an example of how an imported TAAM data set may look in the perspective viewport. It shows roughly 100 flights with a total of 10 000 trajectory events that are arriving at or leaving Oslo.

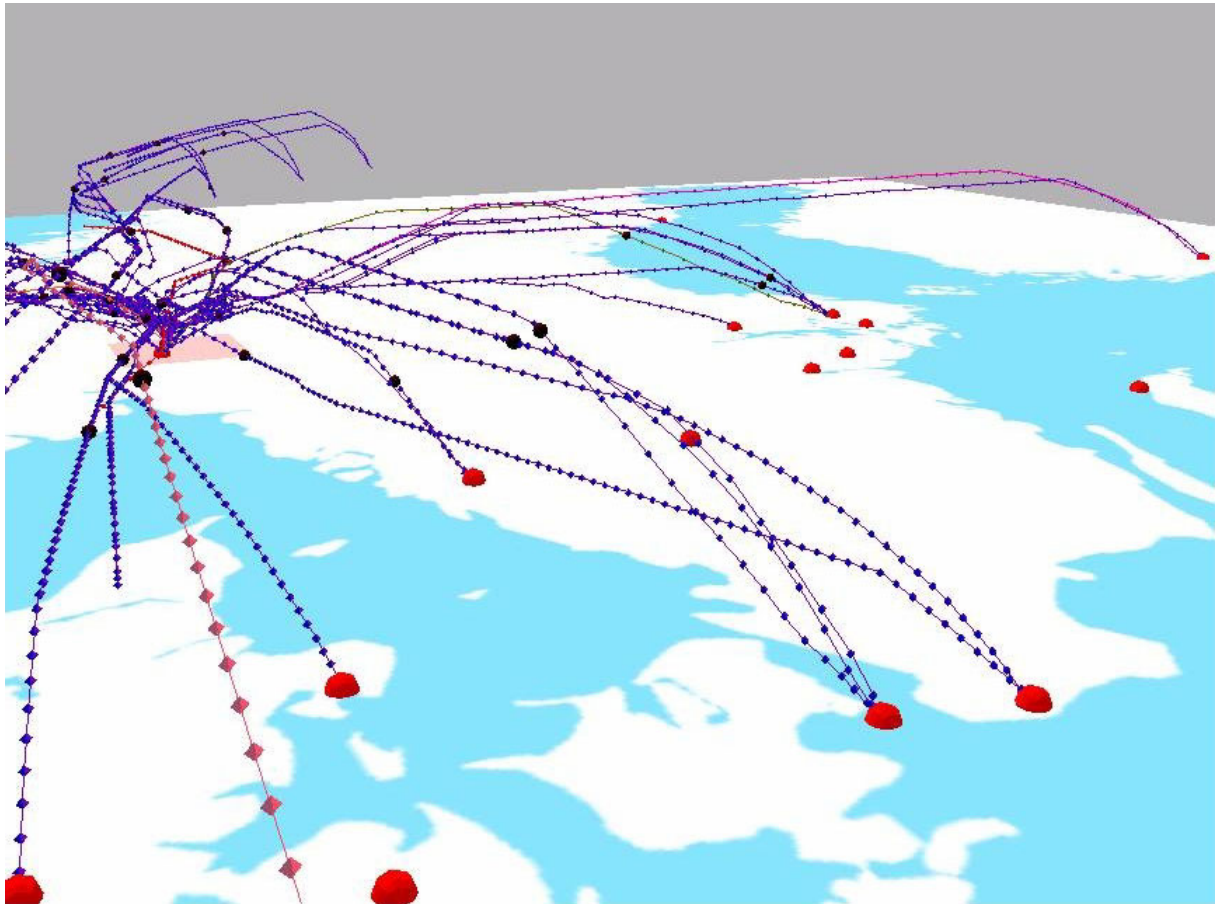


Figure 16 – Example of how an imported data set may look in the perspective viewport

4.4 Finalize and Run Scenario

When a scenario has been created, either manually or through an import, it can be finalized and tested in different ways before it is exported. The options are to:

- add flights
- edit flights
- delete flights
- animate scenario

4.4.1 Add flights

If the user wants to add even more flights to the scenario, the steps in chapter 4.2 should be followed.

4.4.2 Edit flights

There are many ways to edit a flight. To start, the user selects the desired flight in the flight list, or clicks on a flight in any of the graphical viewports (orthogonal viewport, perspective viewport or height profile). As guidance, the flight is highlighted in the graphical viewports by making the line thicker. If the user clicks the Edit button below the flight list, the flight properties dialog window (Figure 9) will open. In this window the flight's callsign, airplane type, departure and destination airports and corresponding time can be modified. Also, the flight's color can be changed so it can more easily be distinguished in the perspective and

orthogonal viewports. Its visibility can also be turned on or off to show only the important flights in the scenario.

Furthermore, the flight's trajectory events can be modified, both with respect to position and ETA (Estimated Time of Arrival). Events can also be added, by following the steps in chapter 4.2, or deleted by clicking the Delete button under the trajectory event list.

There are two ways to modify an existing trajectory event: either by entering values through the keyboard, or by dragging and dropping using the mouse.

Values are modified through the keyboard in the trajectory properties dialog window (Figure 12). This window can be accessed by:

- selecting the trajectory event in the trajectory event list and clicking the Edit button below it
- right clicking a trajectory event icon in any of the graphical viewports

When an event is selected, its icon in the graphical viewports will be highlighted for extra guidance.

The other way to modify a trajectory event is by using drag and drop with the mouse. This can be done in the orthogonal viewport and in the height profile. In the orthogonal viewport, the latitude and longitude can be changed by dragging a trajectory event icon in the y and x directions respectively.



Figure 17 – Moving a trajectory event in the orthogonal viewport

In the image above, a trajectory event belonging to the flight in green color has been moved towards the 60N latitude and slightly eastwards towards the 17E longitude (shown as a dashed green line).

Similarly, a trajectory event can be dragged in the height profile. The height profile's axes are time and altitude in the x and y directions respectively. As guidance for the altitude, the altitude profile has horizontal lines – where each line represents flight levels 0, 100, 200, 300. Flight levels are the regulated altitude levels where aircraft are given clearance to fly.

4.4.3 Delete flights

A flight can be deleted permanently from the scenario. To delete a flight, the user has to select the flight, and click the Delete button under the flight list. This action will delete the flight and all its trajectory events from the scenario data structure.

A safer way to work is often to just make the flight invisible. Then all data about the flight is saved, but the flight is not cluttering the screen and will not be included in an export of the scenario.

4.4.4 Animate scenario

To see how the created scenario behaves, it can be animated over time. This is useful because it is important to see where two aircraft on different flight trajectories are positioned during a certain period of time. To animate a scenario, the time manager (Figure 6) is used.

The time the scenario should be animated from is set by dragging the time slider. The start time, current time, and end time is shown directly above the time slider. When the time has been set, an animation speed can be selected with the speed slider. By default, speed is set to 1x, i.e. real time. It can however be increased to 20x.

The scenario is animated in a loop, where the current scenario time is updated every second based on the current time and speed. When the time has been updated, the graphical viewports are refreshed.

In the graphical viewports, an aircraft's position on its flight trajectory is marked by an icon. In the perspective viewport this icon is a black sphere, and in the orthogonal viewport and height profile it is represented by a black circle.

Since all trajectory events are discrete values, there is a high possibility that the current scenario time in the animation loop is in between two trajectory event times. Therefore, the icon representing the aircraft's position on the flight trajectory must be interpolated between the nearest two trajectory events. The method used is simple linear interpolation.

4.5 Output Scenario

When the user is satisfied with the placing of trajectory events and has arranged the departure times of flights so that they form an interesting scenario, all data can be exported into the LSAE specific data format which is mentioned in chapter 3.1.2.

Although there may be many flights in the scenario, only the visible ones are exported. This allows the user to export groups of flights or use greater datasets where not all of the flights are relevant to the scenario being built.

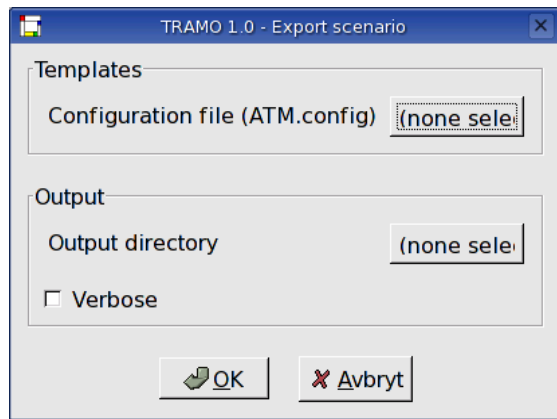


Figure 18 – The export scenario dialog window

In the export scenario dialog window the user first selects which configuration file to edit. This is where the number of flights and flight types are set. The specified output directory will hold the rest of the data files. The directories can be directories shared from the LSAE. It is also possible to manually transfer the files. The files will fit on a 3.5 inch floppy disk, but oftentimes FTP (File Transfer Protocol) is more convenient.

5 Tests

5.1 Functionality Tests

The modular approach of TRAMO enables individual tests of each class. All classes for internal data representation mentioned in Appendix 2 were implemented in a test class for extensive individual tests.

This approach proved to be effective as development of the graphical user interface and the object hierarchy could be detached and programmed individually and thereafter merged gradually. Another benefit of this approach was the functions of the various classes could be verified and thus eliminating complex and potentially time consuming error sources when debugging the graphical components.

5.2 User Tests

Once during the pre-study and twice during the development phase an ATCO was asked to submit his comments. Raymond Dowdall has 20 years of experience as an ATCO at Dublin airport and has been working as a Senior Expert at EUROCONTROL for 15 years. Initially he carried out fast time simulations using the EAM (Eurocontrol Airspace Model) and RAMS model. Now he is working on a large European Community project involving the aviation industry, airline companies, various ANSPs (Air Navigation Service Provider) and EUROCONTROL called C-ATM (Co-operative ATM).

There are several benefits of working with a representative from the targeted user group as opposed to relying fully on fictional user profiles described in the software design document. Immediate feedback to interface implementations and further adjustments to existing procedures is one; clarifications on questions inherent to the aviation industry domain is another.

Initially it was planned to conduct a controlled test with test results deducted from objective test parameters. Such parameters could have been completion time, correctly positioned trajectory events or the ability to correctly give a flight's position at a particular time. These tests were however not carried out. Partly because of shortage of time, partly because there is little or no data collected in a similar environment to produce relevant comparisons and deduct useful results. Hence the final test consisted of Dowdall constructing a scenario which was later successfully exported to, and run in, the LSAE.

During the final test the application was deemed manageable and served its purpose. However, Dowdall made some recommendations that could further increase the manageability of the application. The recommendations were to:

- express altitudes in flight levels instead of feet
- implement functions for copying and pasting flights
- remove the possibility to move trajectory events temporally in the height profile
- make it possible to enlarge orthogonal viewport so it covers the entire display

Dowdall would like all altitudes to be expressed in flight levels rather than feet. The reason for this is that ATCOs use the terminology flight level rather than hundreds of feet.

Furthermore, Dowdall suggested features that resemble functions for copying and pasting data in a Windows operating system environment. If several flights were to be constructed a time saving feature would be the ability to copy a new instance of a flight and edit its separate list of trajectory events. The same feature would be useful when creating a large number of trajectory events for a particular flight, as this task tends to become tedious when done manually.

Dowdall also mentioned that the possibility to change the time of occurrence of a trajectory event in the height profile can be confusing. He questioned the general idea of having both temporal and spatial domains in the same diagram. This will be discussed further in chapter 7.1.

Finally, Dowdall could see use in a function that magnifies the orthogonal viewport so that it covers the entire display. This is interesting since this very much resembles the two-dimensional way ATCOs work today.

These suggestions should be taken into account before a future revision of TRAMO. Ideally such a revision should be accompanied by a simpler task analysis for plausible tests.

6 Results

6.1 Implementation Goals

The following chapter recapitulates the implementation goals listed in chapter 3.2, Table 1, and assesses to what extent they have been achieved. The result of each goal is listed with its corresponding implementation goal ID. If further explanation is needed, references are given to the appropriate chapters or figures.

6.1.1 Must

- ID 1 Data can be exported from TRAMO to LSAE using the functions described in chapter 4.5.
- ID 2 TRAMO allows validated data to be imported from RAMS and TAAM. CFMU and XML data was not implemented in the application due to reasons described in chapter 4.3.
- ID 3 Trajectory events have free positioning in 3D and time. The only restriction is that the trajectory events must be positioned within the geographical region specified in chapter 3.6.1.
- ID 4 A scenario, with all its flights and trajectory events, can be created manually using the methods described in chapter 4.2.
- ID 5 TRAMO covers the geographical area described in chapter 3.6.1 which includes the essential parts of the LSAE geographical area. The differences are deemed to have little or no importance and do not justify the work of searching for new geographical data.
- ID 6 TRAMO is built using a modular approach, where functions are grouped together in specific classes in the code. If a specific function needs to be changed; only the related class has to be modified and recompiled, while the rest of the application can be left intact. Some of this modularity is described in detail in chapter 4.1, while the code structure and class definitions are found in chapter 3.5.

6.1.2 Should

- ID 7 The scenario can be animated and previewed before export using the methods described in chapter 4.4.4.
- ID 8 The application has been deemed manageable by an ATCO after performing the user tests in chapter 5.2. Some recommendations were made by the test subject in order to further improve the manageability of the application, and these are listed in the same chapter.
- ID 9 Airports are displayed with icons, but their names are only shown in the orthogonal viewport.

- ID 10 The work necessary to display air routes and waypoints was not justified due to the reasons explained in chapter 4.2.

6.1.3 Preferable

- ID 11 In order to send the data from TRAMO to LSAE via a direct data feed, it was found that it would require modifications on how LSAE reads data. Modifications to the LSAE source code are out of scope of this project according to chapter 3.6.5.
- ID 12 The function of having TRAMO work as an intermediary for direct data feed from flight simulator software to LSAE would, as ID 11 above, require modifications to the LSAE source code. Therefore it could not be implemented.

6.2 User Interface Requirements

The following chapter recapitulates the interface requirements listed in chapter 3.4.2, Table 4, and assesses to what extent they have been achieved together with their corresponding ID. If further explanation is needed, references are given to the appropriate chapters or figures.

6.2.1 Must

- ID 4 Airports positions are updated in all graphical viewports when time slider moves in the time manager (Figure 6). This procedure is described in chapter 4.4.4.
- ID 5 The actual time of the slider position is shown above the time slider.
- ID 7 There are buttons for adding, editing and deleting flights. These are located below the flight list.
- ID 8 The trajectory visibility can be switched on and off in the flight property dialog window (Figure 9).
- ID 10 The trajectory list is always sorted according to time of occurrence. This is done already when the list is created, as the events are placed in the list according to when they occur. This is further discussed in chapter 7.1.2.
- ID 12 The scenario can be navigated and zoomed in the orthogonal viewport using the methods described in chapter 4.1. Zooming is further discussed in chapter 7.1.1.
- ID 13 The coordinate grid, along with the corresponding latitude and longitude coordinate markings is displayed in the orthogonal viewport. An example of this is shown in Figure 17.
- ID 18 In the perspective viewport, only 3 DOF is supported. The user can rotate around the center point of the ground plane. The angle from the plane is fixed, but the distance from the center point can be changed by zooming in and out.
- ID 22 The altitude of a trajectory event can be edited by dragging it in the y direction of the height profile. This procedure is described in further detail in chapter 4.4.2.

6.2.2 Should

- ID 1 Each interface in TRAMO is automatically resized when the main window is resized (i.e. when the user changes size of the application window by either dragging its edge or using the maximize or minimize functions). This is handled automatically in wxWidgets.
- ID 6 There are two buttons for starting and stopping the animation. These are located in the leftmost part of the time manager. Their specific function is described in chapter 4.4.4.
- ID 9 The flight list does not sort alphabetically, but according to when the flight was added. The first flight that was added to the scenario is at the top of the list, the last at the bottom.
- ID 11 There are buttons for adding, editing and deleting trajectory events. These are located below the trajectory event list.
- ID 14 There is no way to default the orthogonal viewport if the user gets lost. This was considered unnecessary since there is a possibility to zoom out of the viewport, and thereby regain overview.
- ID 15 The trajectory events can be edited by dragging and dropping them as described in chapter 4.4.2.
- ID 19 As in the orthogonal viewport, no default viewport function is implemented in the perspective viewport. Also, when only 3 DOF is provided, the chance is smaller that the user will get lost.
- ID 20 The grid with corresponding latitude and longitude coordinate markings is not shown in the perspective viewport. For the grid to be useful, there must be a projection of the objects in 3D space onto the ground plane, or else the user will experience difficulties to correctly interpret the perspective viewport. One way to solve this perspective problem and make such a grid useful is to implement “droplines”, which are straight lines connecting objects in 3D space with their projected position onto the ground plane. In everyday life, shadows serve this purpose. Droplines were however not implemented due to time shortage, and therefore not the coordinate grid.
- ID 21 A rectangle is shown in the perspective viewport, corresponding to the currently visible area in the orthogonal viewport. This is illustrated in Figure 4.
- ID 23 In the height profile, there is no possibility to scale or zoom. This was considered unnecessary since the interface itself was enlarged from the initial sketch (Figure 2). The interface now covers the whole upper part of the framework (Figure 4).
- ID 24 Since no zooming and scaling was implemented, there was no need to implement a function to return to the default view.

- ID 25 The altitude of a trajectory event can be changed using drag and drop in the y direction, as described in chapter 4.4.2.

6.2.3 Preferable

- ID 2 A debug window was not implemented. The purpose of the debug window was to print information to the user in the case of program malfunction or invalid user handling. This is however handled by other methods: through pop-up alerts when the user performs an invalid action, or through the terminal window if the application malfunctions.
- ID 3 The individual interfaces can not be resized individually. In order to have individual resizing of windows, splitter classes needs to be implemented. The wxWidgets splitter classes had poor usability, and were therefore excluded.
- ID 16 The work necessary to display air routes and waypoints was not justified due to the reasons explained in chapter 4.2.
- ID 17 The possibility to add and delete trajectory events in the orthogonal viewport was not implemented because of time shortage.
- ID 26 The possibility to add and delete trajectory events in the perspective viewport was not implemented because of time shortage.

6.3 Performance

Obviously there is a multitude of applications that have been developed for parsing and interpreting data. However, performance figures seldom hold any relevance since the parsed and interpreted data can be of such varying structure. Furthermore, data processing often have highly specialized functions that make strict comparisons impossible.

As mentioned in chapter 3.6.6, TRAMO handles a sufficient amount of data. TAAM imports have been done where the parser has been asked to process 200 000 entries, interpreting every tenth entry. If each flight contained 100 trajectory events on average, this corresponds to 200 flights.

6.4 Purpose Fulfilment

The purpose of this project has been fulfilled. TRAMO allows users to model arbitrary scenarios and export them into the LSAE environment. Furthermore it features the possibility to import scenarios that have been simulated in professional simulator software, where realistic and validated scenarios can be created.

6.5 Applicability

The design principles used in TRAMO can be used for data visualization and data manipulation in general. The software can accept data from both specific (RAMS, TAAM) and general (XML) data sources. The data can be reduced by selecting every nth sample point, and further minimized by exporting only the flights that are made visible in the scenario.

7 Discussion

7.1 Findings

7.1.1 Zooming in Scenario

It may be the case that the user would like to import a data set where the sample points, the trajectory events, are located very closely. Such a data set would be said to have a high resolution, high detail, because there is a lot of data, many trajectory events, describing a trajectory.

When the user zooms out of the scenario there is no longer any need to draw all trajectory events in order to describe the trajectory.

One way of determining the level of detail of the drawn trajectory is to spatially subdivide the airspace through which the trajectory extends. The algorithm will have to continue subdividing the airspace into smaller and smaller cubes until only one trajectory event is fitted into each cube.

If a cube is said to fit eight smaller cubes, which in their turn fit eight cubes, the hierarchy can be described with a node tree structure where each node has eight children. This subdivision is commonly referred to as an octree.

A convenient way of determining how many trajectory events to draw in the scenario depending on zoom factor would be to traverse the tree to a certain depth proportional to the zoom level. Naturally the trajectory's first and last trajectory event always has to be included (which conveniently happens to be an Airport object that inherits the Coordinate class before placed in the Flight object).

The proportionality between the zoom factor and the depth should either be adjustable or be exponential in such a way that more and more trajectory events are removed from the viewport in the outer zoom positions. This function will be an integral part from a usability perspective. One would not like to see the shape of the curve and want to adjust it but be unable to reach a trajectory event since it has been placed too far down in the octree structure and therefore is deemed obsolete.

The same approach could be used to speed up the drawing of trajectories in orthogonal viewport. Two dimensions would only call for a quadtree, which could have been achieved through simple modifications of the previous octree class.

To conclude, an octree would further optimize data handling and display. The data structure is prepared, but the interaction consequences are not fully investigated.

7.1.2 Sorting Lists

The most effective solution to a problem is often to avoid the problem rather than solving it. Instead of sorting a dataset just before it is going to be used, it might be a good idea to sort it when it is being built so that each record gets into its correct place already from the

beginning. This way there is no need to make a decision on which sorting algorithm to use. All operations are done with proportional time, or time complexity $O(n)$ if one prefers.

When a new trajectory event is added it is inserted chronologically. However, some datasets, such as the ones from TAAM and RAMS, are sorted by time already. Hence there is no need for a bulk sorting algorithm.

7.1.3 Spatial and Temporal Axes

In the perspective viewport the scenario is drawn in three spatial dimensions. As the scenario is animated, one can claim that the scenario stretches over four dimensions where time is considered being the fourth axis.

The orthogonal viewport has two spatial dimensions. When animated, the third dimension is time.

The height profile also has two dimensions. One of the axes represents altitude or flight levels. The other axis represents time. Contrary to the other viewports, no third dimension is introduced in the height profile when the scenario is animated. The intended use for this interface is for the user to be able to adjust a flight so that it reaches a certain altitude after certain amount of time. Test subject and expert Dowdall often uses this spatial-temporal relation to assert if a flight behaviour is realistic or not. Therefore it is believed that this viewport may be useful for plotting performance curves.

However, the spatial-temporal relationship causes confusion as well. If a trajectory event is dragged up or down in the viewport this will represent a change of altitudes. But how shall a movement sideways be interpreted? Currently TRAMO interprets this interaction as a certain altitude being reached at the time where the trajectory event is dropped, however the spatial position of the trajectory event is not changed. Since only time is changed, it is in effect the order in the trajectory events' connection list that is being altered. A trajectory event's position remain the same, but the lines interconnecting them, in essence making up the route, is no longer connecting to its nearest neighbour.

To conclude, although there is a possibility of mixing dimensions in an interface so that it becomes useful in that particular viewport performing a specific task, the manipulation might force the user to have to work in two or more interfaces in order to make all necessary adjustments. Therefore the recommendation must be to arrange each interface so that they handle either spatial or temporal dimensions. Graphical interfaces shall be arranged so that user interaction only manipulates dimensions of similar kind. (The argument can be extended to any dataset where one vector contains another vector, RGB values in a volume for instance. Is there really a use for changing the "redness" of all voxels along the YZ-plane $X=1$?)

7.2 Future Development

Apart from the goals and requirements that for various reasons were not implemented, this chapter describes possible future development of TRAMO.

7.2.1 Interaction Model

The current interaction model should be refined to capture the intuitive user gestures. This calls for a user study and borders to disciplines such as Human Factors. For example, the perspective viewport could be navigated through mouse interaction, eliminating frequent use

of the keyboard. The navigation model itself needs improvement. Which model that is the most beneficial and easy to use could be the scope of a future project (not only including TRAMO, but also other 3D environments)?

7.2.2 Platform

The internal data representation is generic in terms of platform. So is the intermediary code. Challenges arise when trying to construct a uniform graphical user interface for multiple platforms. The wxWidgets API documentation promises cross-platform compatibility.

With a few adjustments and a new compiler it should be fairly straight forward to create a version for both Windows and Macintosh platforms.

7.2.3 New Output Format

The modular approach of the TRAMO source code makes it possible to produce new output units. As long as the conceptual model of traffic is somewhat similar and the mandatory data has been loaded into TRAMO, it should only be a matter of adhering to the new file standards.

This will make TRAMO a useful tool for creating scenarios to any new ATM prototype environment.

8 Glossary

8.1 Abbreviations

ACARE	Advisory Council for Aeronautical Research in Europe is an organization with members representing the manufacturing industry, airlines, airports, service providers, regulators, the research establishments and academia focusing on establishing and implementing a strategic research agenda. The strategic research agenda is updated every two years.
AIP	Aeronautical Information Publication is published by a country's civil aviation administration and contains national regulations and requirements, national holidays, chart symbols, measurement units, navigation aids etc.
ANSP	Air Navigation Service Provider, such as Air Traffic System Reporting Offices and Towers.
API	Application Programming Interface is a software library where common functions are arranged in such a way that they can be more easily used by a software developer.
ASCII	American Standard Code for Information Interchange is the most common format for text files which allows for 128 characters each made up of 7 bits.
ATC	Air Traffic Control
ATCC	Air Traffic Control Center
ATCO	Air Traffic Controller
ATM	Air Traffic Management
BADA	Base of Aircraft Data is an aircraft performance database. It provides a set of ASCII files containing performance and operating procedure data for different aircraft types. The BADA aircraft database is used to provide the necessary data on aircraft performance and operations models, which are suitable for trajectory calculation and prediction within ATC simulations. The aircraft models are based on a Total Energy Model (TEM).
C-ATM	Co-operative ATM
CFMU	Central Flow Management Unit is operated by Eurocontrol and provides airline companies with a flight plan for every flight in the European airspace.

DOF	Degrees Of Freedom is the number of axes it is possible to rotate around and translate along in a virtual environment. In a 3D-environment there are 3 axes, and thus maximum freedom is achieved when it is possible to rotate around all three axes (“3 DOF”) and simultaneously translate along all three axes (“3 DOF”). Therefore maximum movement in 3D is called “6 DOF”.
DOM	A Document Object Model is a node tree hierarchical data structure which XML data can be parsed into. It facilitates searching and manipulation by using recursion.
EAM	Eurocontrol Airspace Model
EEC	EUROCONTROL Experimental Center
EPHE	Ecole Pratique des Hautes Etudes
ETA	Estimated Time of Arrival
FTP	File Transfer Protocol is used for transferring files between computers.
HCI	Human Computer Interaction
ICAO	International Civil Aviation Organization
INO	The “innovative research” research area at EUROCONTROL Experimental Center.
LSAE	Linköping Semi-Immersive ATC Environment
px	Pixel, picture element.
RAMS	Reorganised ATC Mathematical Simulator is an ATM simulation software developed by ISA Software that performs fast time simulation on aircraft movement from a macroscopic view down to airport level.
RGB	Red Green Blue are the color components that make up the additive color scheme used in computer graphics. The three components are numbers that indicate the amount of each respective color.
SID	Standard Instrument Departure is a set of instructions for an aircraft to safely take off and leave the airport area and merge with a larger route network airway. A SID is usually named after the navaid marking the entry point to the airway.

SRA	The ACARE Strategic Research Agenda.
STAR	Standard Terminal Arrival Route are instructions to exit a route network airway heading for a waypoint from which the approach to the airport is made.
TAAM	Total Airspace and Airport Modeler, simulation tool for airspace scenarios developed by the Boeing company Preston Solutions.
TRAMO	Traffic Modeler
UML	Unified Modeling Language
UTF-8	UCS Transformation Formats allowing multi-octet characters. UTF-8 preserves the full ASCII range and provides compatibility with systems based on ASCII values but is also transparent to other values.
VR	Virtual Reality is the simulated real or imagined environment experienced in all three dimensions, and possible also time, through computer graphics.
XML	eXtensible Markup Language

8.2 Terminology

Callsign	Unique naming for both private and commercial aircraft. In the latter case the letters typically denotes the airline company. “SK123” would be Scandinavian Airline Systems and “UA456” United Airlines.
Dropline	Straight line connecting object in 3D space with its projected position onto the ground plane.
Flight level	A flight level (FL) is a regulated altitude level where an aircraft is given clearance to fly. FL’s are numbered 0, 10, 20...n. The corresponding altitude in feet is $FL * 100$. This implies that there is 1000 ft clearance between each flight level.
Holdstack	A holdstack is two or more aircraft circling a designated safe area, separated by different altitude clearances, waiting to commence final approach and land at an airport.
Navaid	Navigational Aid
Pushback	The action performed by a ground vehicle pushing a jet aircraft out of its gate position onto its taxiway towards the takeoff runway.
Stereoscopic vision	Produced by using shutter glasses that, based on LCD display technique, alternate blocking the sight of right and left eye. An IR

signal synchronizes the shutters with alternating images on the screen adjusted in perspective as to create a perceived sensation of image depth.

Tracking system

Equipment intended to assert the position of the user's head in order to establish the line of sight and derive perspective. The tracking can also apply to any auxiliary object which should be present in both the real and virtual reality, such as a "wand" or a "glove".

Voxel

Volume element.

9 Bibliography

- [1] B. Stroustrup, *C++ Programming Language Third Edition*, Addison-Wesley, Reading (Massachusetts, USA), 1998.
- [2] OpenGL - The Industry Standard for High Performance Graphics, 2004-11-02
<http://www.opengl.org/>
- [3] wxWidgets Home, 2004-11-02
<http://www.wxwidgets.org/>
- [4] Trolltech – Qt Product Overview, 2004-11-02
<http://www.trolltech.com/products/qt/index.html>
- [5] wxWidgets Documentation, 2004-11-02
<http://www.wxwidgets.org/docs.htm>
- [6] EUROCONTROL - European Organisation for the Safety of Air Navigation, 2004-11-02
http://www.eurocontrol.int/corporate/public/standard_page/org_ourorganisation.html
- [7] EUROCONTROL - Eurocontrol Experimental Centre, 2004-11-02
<http://www.eurocontrol.int/eec/>
- [8] EUROCONTROL - Innovative Research, 2004-11-02
<http://www.eurocontrol.int/eec/projects/INO.htm>
- [9] Advisory Council for Aeronautical Research in Europe, 2004-11-02
<http://www.acare4europe.org/>
- [10] T. Dang Nguyen, H. Le-Hong, M. Tavanti, “Visualization and Interaction On Flight Trajectory in A 3D Stereoscopic Environment”. In Proceedings of the *AIAA/IEEE 22nd Digital Avionics Systems Conference*, Indianapolis (Indiana, USA), October 2003.
- [11] T. Dang Nguyen, H. Le-Hong, M. Tavanti, “Empirical Analysis of the Applicability of 3D Stereoscopic in Air Traffic Control”. In Proceedings of *IEEE 6th ITSC2003, International Conference on Intelligent Transportation Systems*, Shanghai (China), October 2003.
- [12] T. Dang Nguyen, H. Le-Hong, M. Tavanti, “Three-dimensional Stereoscopic visualization for Air Traffic Control Interfaces: a preliminary study”. In Proceedings of *AIAA/IEEE 22nd Digital Avionics Systems Conference*, Indianapolis (Indiana, USA), October 2003.
- [13] M. Cooper, V. Duong, J. Hjalmarsson, M. Lange, A. Ynnerman, “Air traffic management project report on progress during phase II”, Linköpings Universitet, Linköping (Sweden), 2004.

[14] M. Cooper, V. Duong, J. Hjalmarsson, M. Lange, A. Ynnerman, “3D Visualization and 3D and Voice Interaction in Air Traffic Management”. In proceedings of *SIGRAD 2003*, Umeå (Sweden), 2003.

[15] RAMS Plus, 2004-11-02
<http://www.ramsplus.com/>

[16] ISA Software, 2004-11-02
<http://www.isa-software.com/>

[17] Preston Aviation Solutions - TAAM Outline, 2004-11-02
<http://www.preston.net/products/TAAMoutline.htm>

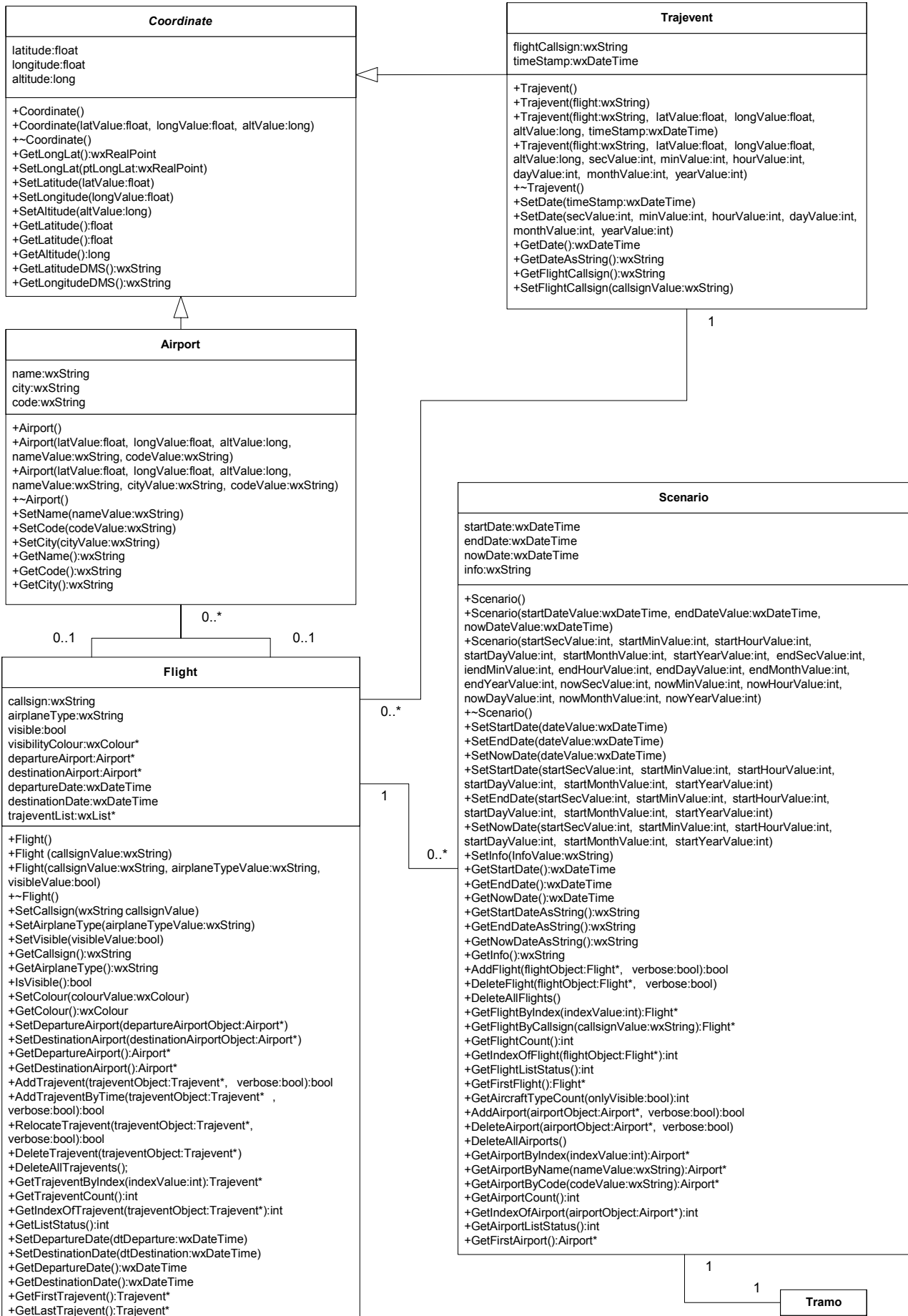
[18] Preston Aviation Solutions, 2004-11-02
<http://www.preston.net/>

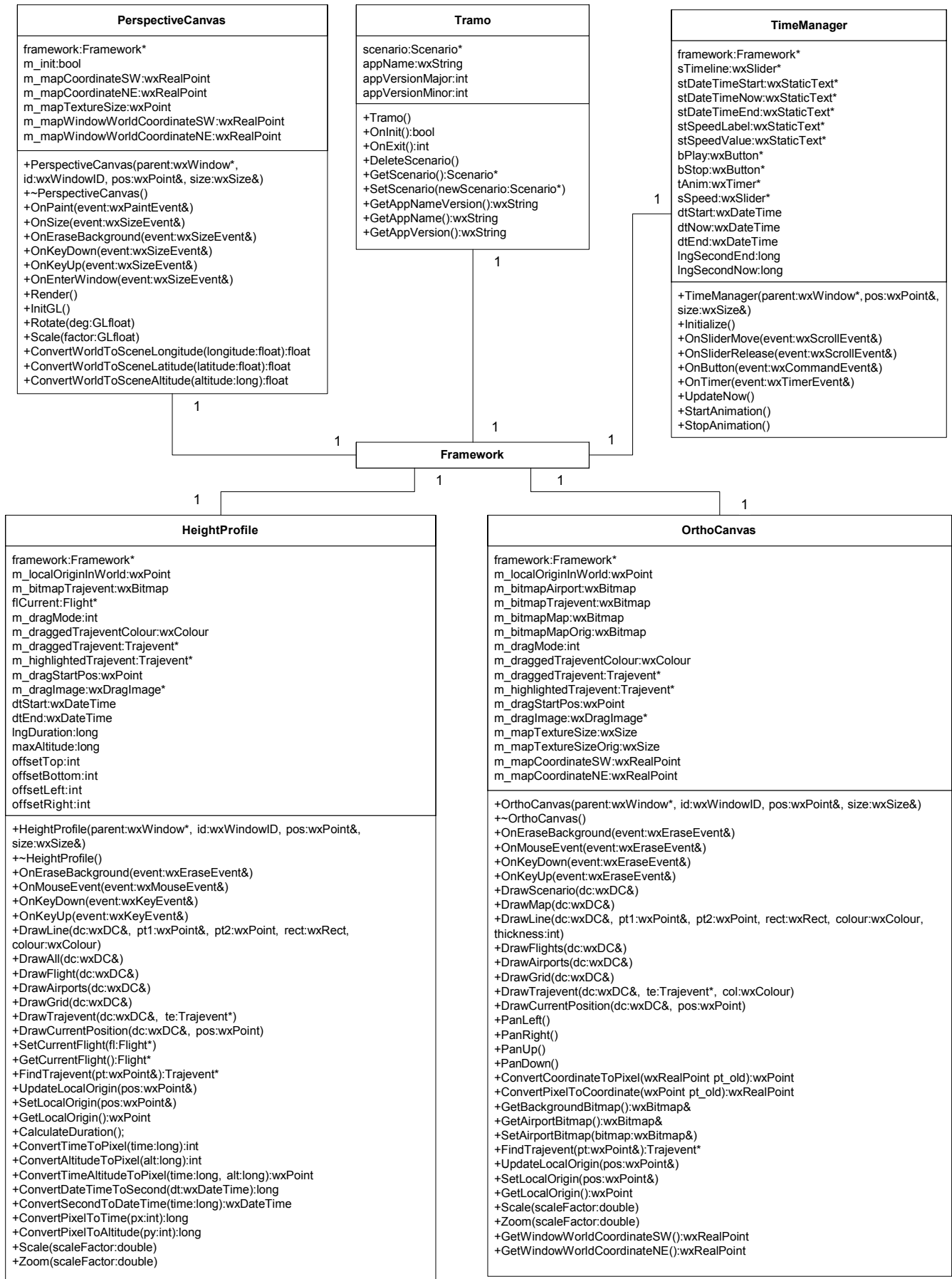
[19] XMLwrapp, 2004-11-08
<http://pmade.org/software/xmlwrapp/>

[20] CFMU Home, 2004-11-11
<http://www.cfm.eurocontrol.be/>

[21] E. Jeannot, C. Kelly, D. Thompson, “The Development of Situation Awareness Measures in ATM Systems”. EUROCONTROL, Paris (France), 2003.

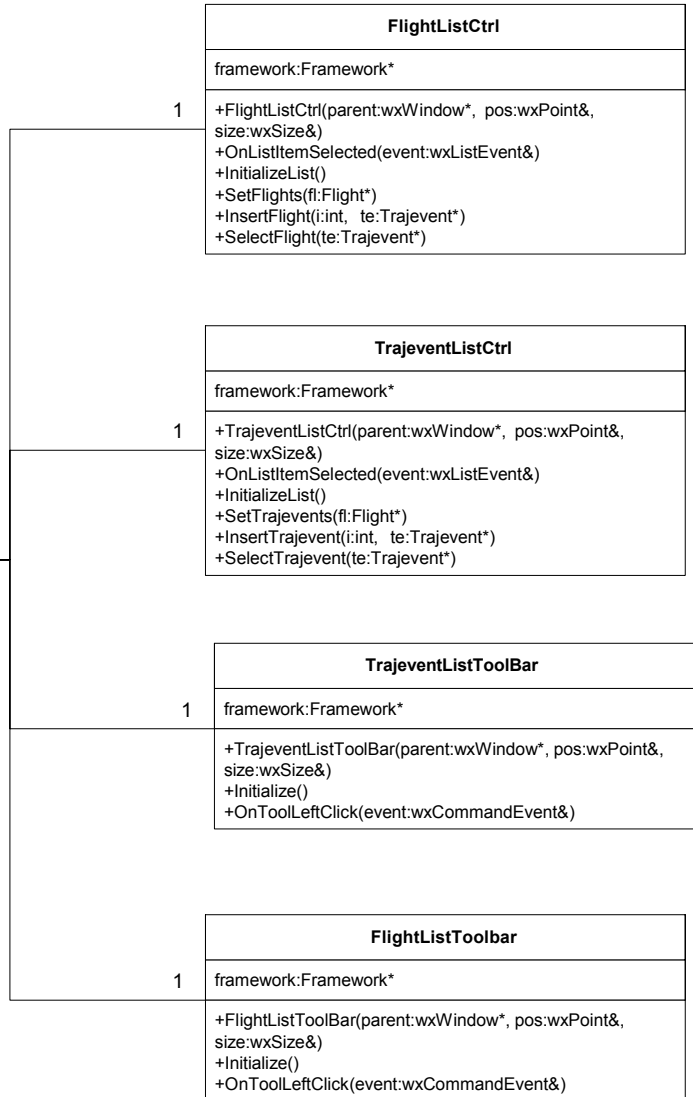
Appendix 1: UML Diagrams

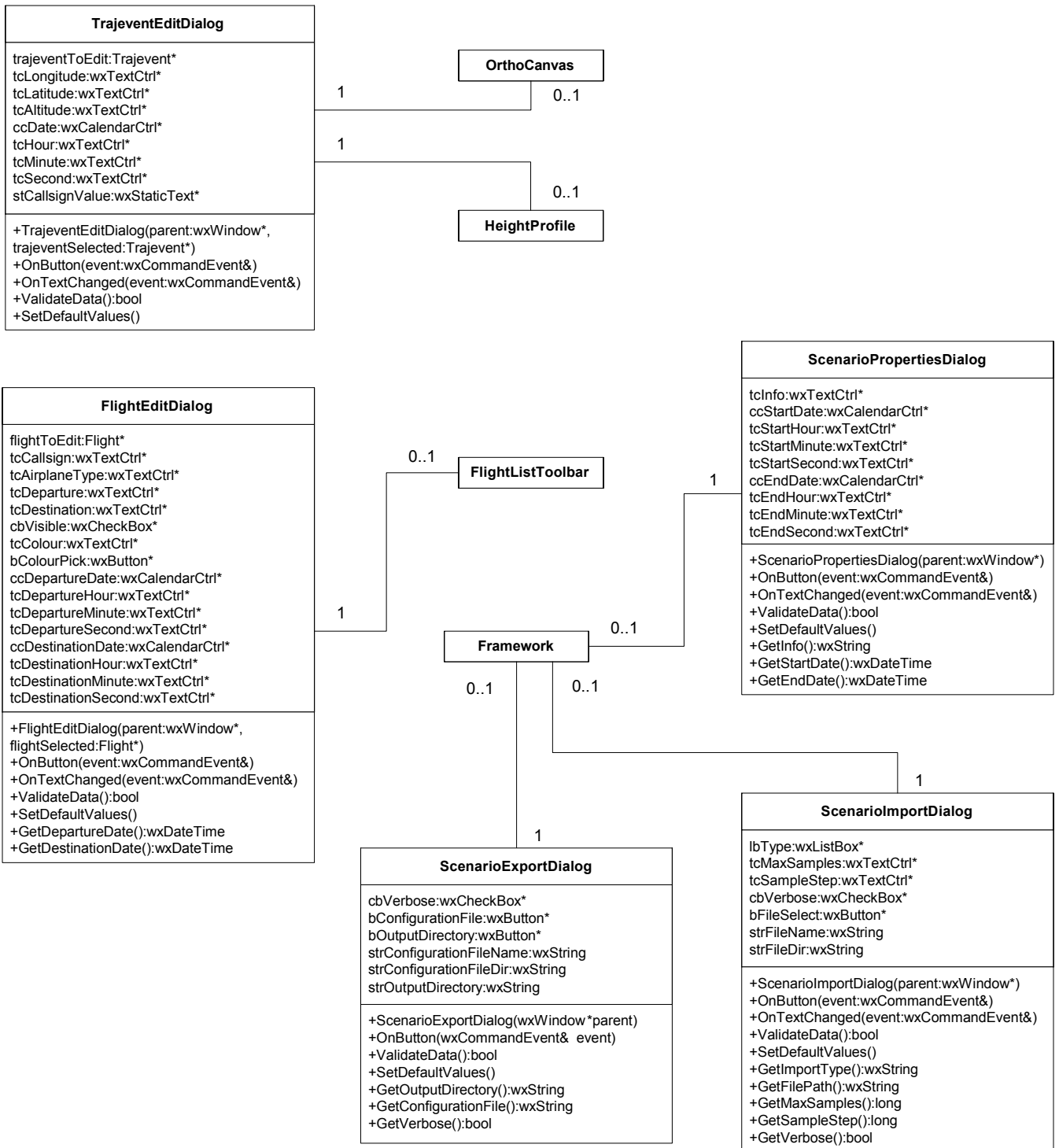




Framework
oc:OrthoCanvas* fltb:FlightListToolBar* flc:FlightListCtrl* pc:PerspectiveCanvas* tlc:TrajeventListCtrl* ttlb:TrajeventListToolBar * stLabelFlights:wxStaticText* stLabelTrajevents:wxStaticText* flSelected:Flight* teSelected:Trajevent* tm:TimeManager* hp:HeightProfile* sbMain:wxStatusBar*
+Framework() +Initialize() +RefreshAll() +RefreshCanvases() +RefreshFlightListCtrl() +RefreshTrajeventListCtrl() +RefreshOrthoCanvas() +RefreshPerspectiveCanvas() +RefreshHeightProfile() +OnAbout(event:wxCommandEvent&) +OnQuit(event:wxCommandEvent&) +OnFullScreen(event:wxCommandEvent&) +PrepareDC(dc:wxDC&) +PrepareClientDC(dc:wxDC&) +SetSelectedFlight(fl:Flight*) +SetSelectedTrajevent(te:Trajevent*) +OnScenarioProperties(event:wxCommandEvent&) +OnScenarioNew(event:wxCommandEvent&) +OnScenarioOpen(event:wxCommandEvent&) +OnScenarioSave(event:wxCommandEvent&) +OnScenarioImport(event:wxCommandEvent&) +OnScenarioExport(event:wxCommandEvent&) +GetOrthoCanvas():OrthoCanvas* +GetFlightListToolBar():FlightListToolBar* +GetFlightListCtrl():FlightListCtrl* +GetTrajeventListToolBar():TrajeventListToolBar* +GetTrajeventListCtrl():TrajeventListCtrl* +GetPerspectiveCanvas():PerspectiveCanvas* +GetStatusBar():wxStatusBar* +GetLabelTrajevents():wxStaticText* +GetLabelFlights():wxStaticText* +GetSelectedFlight():Flight* +GetSelectedTrajevent():Trajevent* +GetTimeManager():TimeManager* +GetHeightProfile():HeightProfile*

1





Appendix 2: Class Descriptions

GUI Classes

Tramo

The Tramo class inherits wxWidgets' wxApp class which represents the Tramo application itself. It is used to get and set application-wide properties and objects, most importantly the Scenario object which is the main internal data representation class.

Framework

The Framework class inherits wxWidgets' wxFrame class. It is used to gather all classes that collectively make up the graphical user interface. It also handles all events that are raised by the graphical user interface, e.g. menu commands. If an action performed in one GUI class manipulates data so that another GUI classes' content is affected the Framework will perform the necessary updates.

OrthoCanvas

The OrthoCanvas class inherits wxWidgets' wxWindow class. It is used to display a 2D top view of the scenario. It handles basic navigation such as zooming and panning. It also converts image measurements to global coordinates in order to return user interaction in terms shared by the internal data representation classes. Furthermore it accepts user interaction for manipulating each trajectory event's longitude and latitude position.

PerspectiveCanvas

The PerspectiveCanvas class inherits wxWidgets' wxGLCanvas class. It is used to display a 3D view of the scenario. PerspectiveCanvas displays OpenGL objects and performs actions such as scenario rotation and scenario scaling. The PerspectiveCanvas also has functions for simple interpolation of continuous aircraft position between discrete trajectory sample points. It also features a red square view guide superimposed on the scenario ground texture to aid the user's top view navigation.

HeightProfile

The HeightProfile class inherits wxWidgets' wxWindow class. It is used to display a 2D side view of a specific aircraft's trajectory. It accepts user interaction for manipulating each trajectory event's altitude and individual ETA.

TimeManager

The TimeManager class inherits wxWidgets' wxPanel class. It is used to display the current time of the scenario and represent the timeframe over which the scenario is played. It updates the current time in the Scenario object created in the Tramo object, accessible for all other time dependent objects. Setting the time can be done manually by dragging the slider. The play-button performs a similar operation as it invokes an instance of the wxTimer class that updates the current time with increments set by a time factor.

FlightListCtrl

The FlightListCtrl class inherits wxWidgets' wxListCtrl class. It displays all Flight objects present in the scenario. Each Flight object is listed with its corresponding callsign, aircraft type, visibility flag and color. When a Flight object is selected, its corresponding trajectory events are listed in the TrajeventListCtrl object below. Furthermore the line that represents the

flight's trajectory is highlighted in the OrthoCanvas, the PerspectiveCanvas and the HeightProfile.

TrajeventListCtrl

The TrajeventListCtrl class inherits wxWidgets' wxListCtrl class. It displays all Trajevent objects linked to the Flight object selected in the FlightListCtrl object above. Each Trajevent object is listed with its corresponding ETA, latitude, longitude and altitude. When selected, the corresponding Flight object is selected and highlighted in the FlightListCtrl object. The trajectory event is highlighted in the OrthoCanvas, the PerspectiveCanvas and the HeightProfile.

FlightListToolBar

The FlightListToolBar class inherits wxWidgets' wxToolBar class. It contains three buttons: New, Edit and Delete. The New action creates a new instance of a Flight object and adds it to the Scenario object by displaying an empty instance of the FlightEditDialog class for the user to fill out. The Edit action display an instance of the FlightEditDialog class populated with the data from the Flight object selected in the FlightListCtrl object. Delete erases the Flight object selected in the FlightListCtrl object from the Scenario object.

TrajeventListToolBar

The TrajeventListToolBar class inherits wxWidgets' wxToolBar class. It contains three buttons: New, Edit and Delete. The New action creates a new instance of a Trajevent object and adds it to the Flight object selected in the FlightListCtrl object by displaying an empty instance of the TrajeventEditDialog class for the user to fill out. The Edit action display an instance of the TrajeventEditDialog class populated with the data from the Trajevent object selected in the TrajeventListCtrl object. Delete erases the Trajevent object selected in the TrajeventListCtrl object from the Flight object selected in the FlightListCtrl object.

FlightEditDialog

The FlightEditDialog class inherits wxWidgets' wxDialog class. It contains a set of input objects. If the aim is to edit a Flight object, a pointer to the Flight object provides access to the values that the Flight object currently contains. If the aim is to create a new Flight object a new Flight object is created and its pointer provides access to the default values.

The FlightEditDialog class also contains two buttons: OK and Cancel. When the OK-button is clicked the OnButton-event is raised that in its turn updates the Flight object's values.

TrajeventEditDialog

The TrajeventEditDialog class inherits wxWidgets' wxDialog class. It contains a set of input objects. If the aim is to edit a Trajevent object, a pointer to the Trajevent object provides access to the values that the Trajevent object currently contains. If the aim is to create a new Trajevent object a new Trajevent object is created and its pointer provides access to the default values.

The TrajeventEditDialog class also contains two buttons: OK and Cancel. When the OK button is clicked the OnButton-event is raised that in its turn updates the Trajevent object's values.

ScenarioPropertiesDialog

The ScenarioPropertiesDialog class inherits wxWidgets' wxDialog class. It contains a set of input objects, and its purpose is to edit the current Scenario object contained in the Tramo class. The ScenarioPropertiesDialog also contains two buttons: OK and Cancel. When the OK button is clicked the OnButton-event is raised that in its turn updates the Scenario object's values.

ScenarioImportDialog

The ScenarioImportDialog class inherits wxWidgets' wxDialog class. It contains a set of input objects, and its purpose is to replace the current Scenario object contained in the Tramo class with a new Scenario object based on imported flight data. It calls the FileHandler class where a specified file is interpreted and the new Scenario object is created.

ScenarioExportDialog

The ScenarioImportDialog class inherits wxWidgets' wxDialog class. It contains a set of input objects, and its purpose is to export the current scenario data to a format that can be interpreted by external applications. It calls the FileHandler class where a pointer to the current Scenario object provides access to the scenario data. The data is saved to specific files in an output directory.

Internal Data Representation Classes

Scenario

The Scenario class is the main internal data representation class. It has one linked list structure that contains all Flight objects, and another that holds all Airport objects in the scenario. It provides functions for manipulating the Flight and Airport objects in these lists, as well as scenario dates and times.

Flight

The Flight class is the class that holds information about a flight. It contains a linked list structure that contains all Trajevent objects. It can also have pointers to one or two Airport objects, representing departure and destination airport. A Flight object must have either both airports set, or at least two trajectory events, to be visible. In the latter case, the flight is considered to be en-route, since the flight does not start or end in the scenario.

Trajevent

The Trajevent class inherits the Coordinate class, and is the class that holds information about a trajectory event. A trajectory event is a discrete sample point on a flight trajectory. It contains information about ETA, as well as the three dimensional coordinate at which it occurs.

Coordinate

The Coodinate class is an abstract base class. It is inherited by both the Trajevent class and the Airport class. It holds data about spatial location in its member variables latitude, longitude and altitude.

Airport

The Airport class inherits the Coordinate class, and is the class that holds information about an airport. It contains data about the airport location, name, corresponding city and its four character ICAO (International Civil Aviation Organization) code.

I/O Classes

FileHandler

The FileHandler class is an independent class that controls reading and writing to disk. It parses and compiles data files according to specific formats depending on data source. Furthermore it helps reading the texture used in the PerspectiveCanvas class and the image used in the OrthoCanvas class.